Liberté • Égalité • Fraternité
RÉPUBLIQUE FRANÇAISE

MINISTÈRE CHARGÉ
DE L'EMPLOI

DOSSIER PROFESSIONNEL (DP)

Nom de naissance Ongaro
Nom d'usage Ongaro
Prénom David
Adresse 16 Villa Eugénie
94120 Fontenay-Sous-Bois

Titre professionnel visé

Concepteur Développeur d'Application

MODALITÉ D'ACCÈS :

☒ Parcours de formation
☐ Validation des Acquis de l'Expérience (VAE)

Présentation du dossier

Le dossier professionnel (DP) constitue un élément du système de validation du titre professionnel. **Ce titre est délivré par le Ministère chargé de l'emploi.**

Le DP appartient au candidat. Il le conserve, l'actualise durant son parcours et le présente **obligatoirement à chaque session d'examen**.

Pour rédiger le DP, le candidat peut être aidé par un formateur ou par un accompagnateur VAE.

Il est consulté par le jury au moment de la session d'examen.

Pour prendre sa décision, le jury dispose :

1. des résultats de la mise en situation professionnelle complétés, éventuellement, du questionnaire professionnel ou de l'entretien professionnel ou de l'entretien technique ou du questionnement à partir de productions.
2. du **Dossier Professionnel** (DP) dans lequel le candidat a consigné les preuves de sa pratique professionnelle
3. des résultats des évaluations passées en cours de formation lorsque le candidat évalué est issu d'un parcours de formation
4. de l'entretien final (dans le cadre de la session titre).

[Arrêté du 22 décembre 2015, relatif aux conditions de délivrance des titres professionnels du ministère chargé de l'Emploi]

Ce dossier comporte :

- pour chaque activité-type du titre visé, un à trois exemples de pratique professionnelle ;
- un tableau à renseigner si le candidat souhaite porter à la connaissance du jury la détention d'un titre, d'un diplôme, d'un certificat de qualification professionnelle (CQP) ou des attestations de formation ;
- une déclaration sur l'honneur à compléter et à signer ;
- des documents illustrant la pratique professionnelle du candidat (facultatif)
- des annexes, si nécessaire.

Pour compléter ce dossier, le candidat dispose d'un site web en accès libre sur le site.

http://travail-emploi.gouv.fr/titres-professionnels

Sommaire

Pratique professionnelle

Développer la partie front-end d'une application web sécurisée p. 5

- Installer et configurer son environnement de travail ..... p. 5
- Analyser les besoins et maquetter les interfaces utilisateur ..... p. 17
- Définir l'architecture logicielle ..... p. 25
- Développer les interfaces utilisateur ..... p. 30
- Préparer et exécuter les tests ..... p. 36
- Préparer et documenter le déploiement ..... p. 41
- Mettre en production (DevOps) ..... p. 45

Développer la partie back-end d'une application web sécurisée

- Gérer le projet ..... p. 49
- Définir l'architecture logicielle ..... p. 53
- Mettre en place la base de données relationnelle ..... p. 58
- Développer les composants d'accès aux données SQL ..... p. 64
- Développer les composants métier ..... p. 69
- Préparer et exécuter les tests ..... p. 75
- Préparer et documenter le déploiement ..... p. 80
- Mettre en production (DevOps) ..... p. 85

Titres, diplômes, CQP, attestations de formation *(facultatif)* p. 89

Déclaration sur l'honneur p. 90

Documents illustrant la pratique professionnelle *(facultatif)* p.

Annexes *(Si le RC le prévoit)* p.

PRATIQUE PROFESSIONNELLE

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 1 Installer et configurer son environnement de travail

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Dans le cadre du développement de mon projet TaskFlow Manager, j'ai développé deux applications séparées, une API REST en Symfony (backend) et une application web en React (frontend) qui consomme cette API.

Avant de commencer à coder, j'ai préparé mon environnement de travail adapté à ces deux projets, avec trois objectifs :

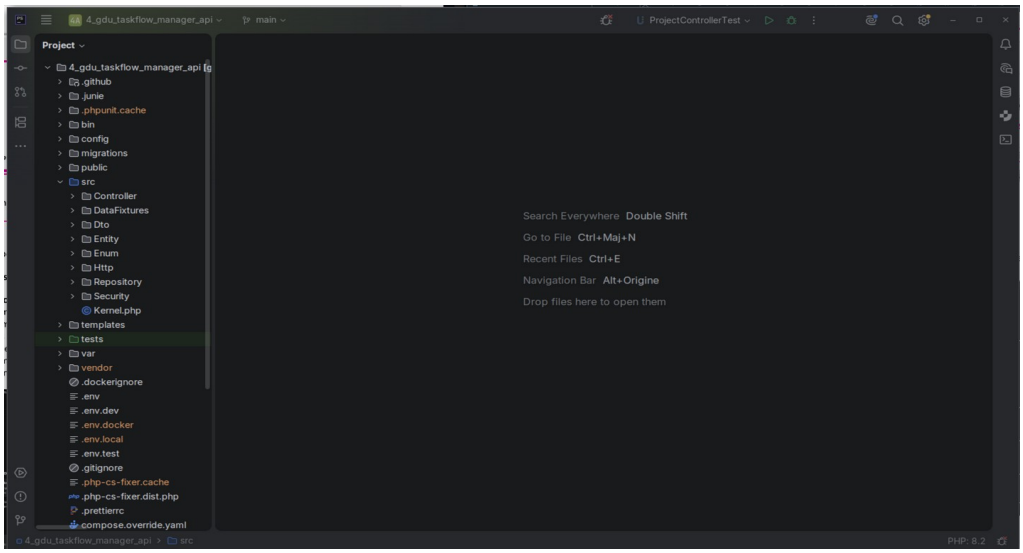
- Travailler vite et efficacement ;
- Garder un code propre ;
- Pouvoir lancer le projet de la même façon sur n'importe quelle machine.

Certains outils sont communs aux deux projets. D'autres sont propres à chacun, car les besoins d'une API PHP et d'un front JavaScript sont différents.

Installation d'outils de développement :

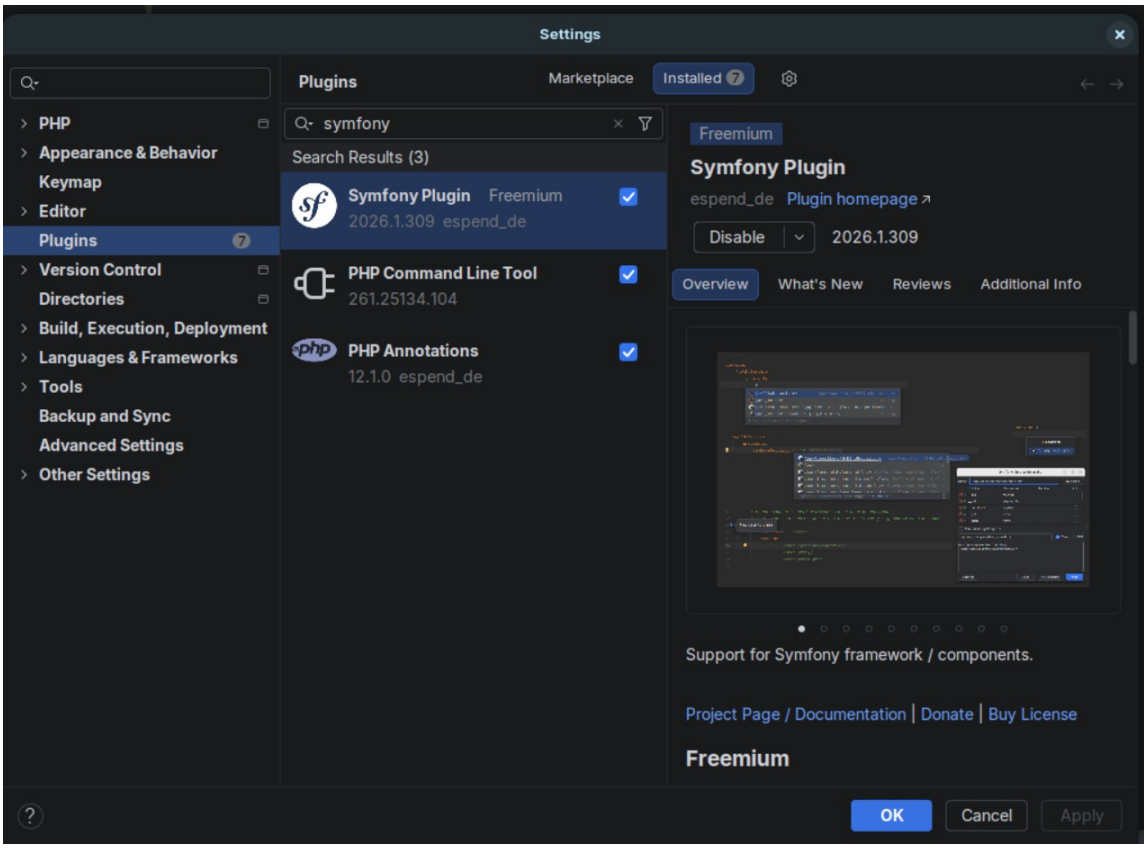
PhpStorm :

J'avais déjà l'habitude de travailler avec PhpStorm et je possède une licence personnelle, je l'ai donc utilisé pendant toute la formation. PhpStorm gère très bien le PHP (autocomplétion, refactoring, navigation dans le code…) et aussi le JavaScript/React. Ce qui m'a permis de travailler sur les deux projets.



Interface PhpStorm

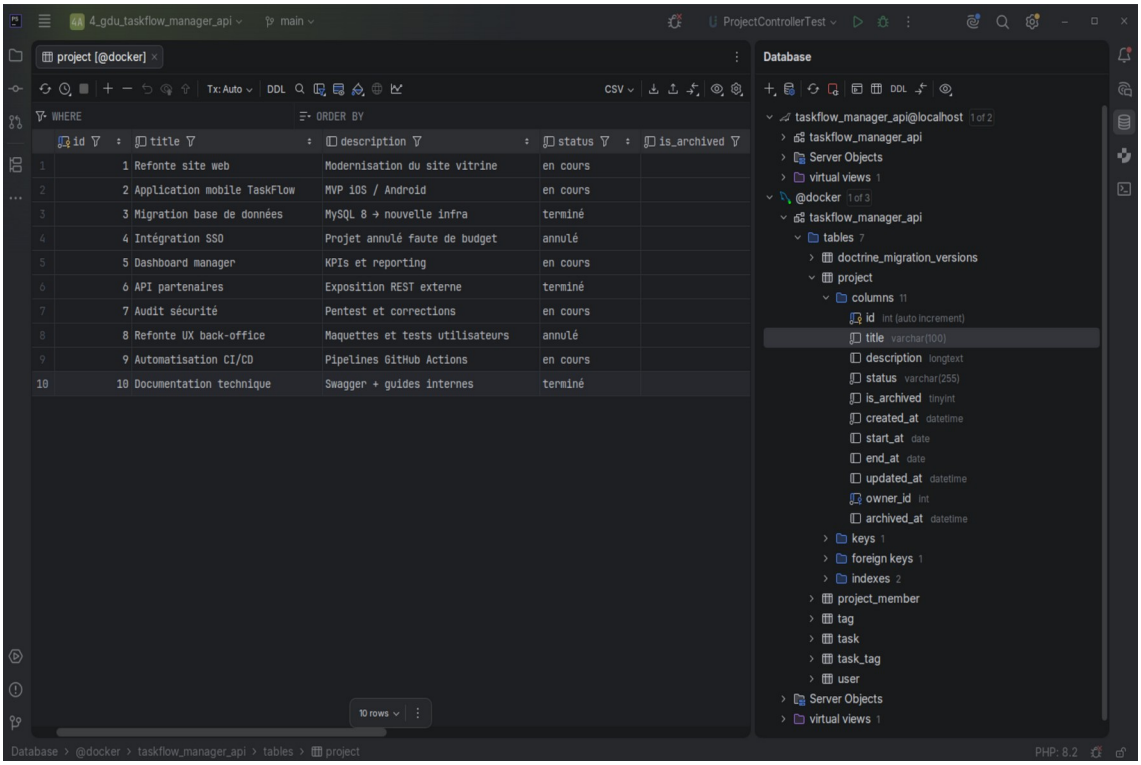
J'ai également installé le **Symfony Plugin** qui ajoute de l'autocomplétion sur les routes, les services, les fichiers de configuration YAML, les entités Doctrine…



*Plugin **Symfony Plugin** installé*

Quelques autres configurations dans PhpStorm :

- Interpréteur PHP 8.2 ;
- PHPUnit pour lancer les tests directement depuis l'IDE ;
- PHP CS Fixer et PHPStan comme outil de qualité, pour voir les erreurs pendant le développement ;
- Outil Database de mon IDE connecté à ma base MySQL pour voir mes tables dans mon IDE.



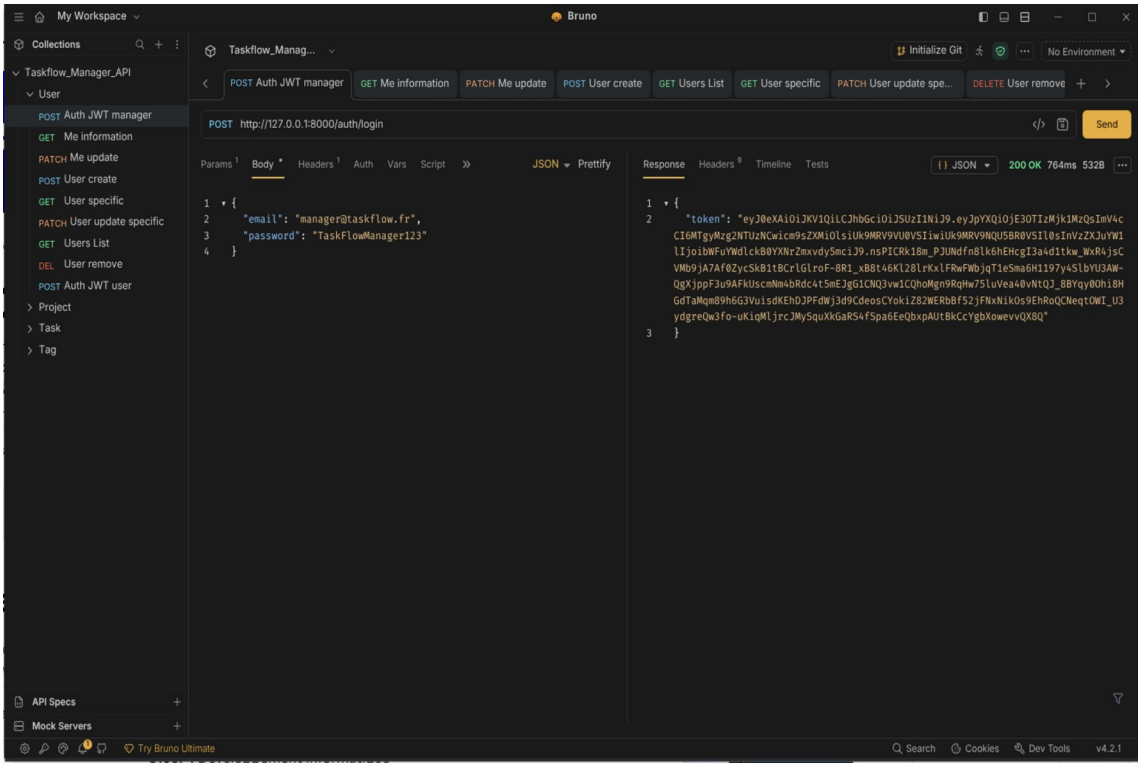
Panneau Database de PhpStorm connecté à la bdd du projet taskflow_manager_api

Les outils installés sur mon poste :

- PHP 8.2 avec les extensions nécessaires (pdo_mysql, zip, ctype, iconv) ;
- Composer 2 pour gérer les dépendances PHP ;
- Symfony CLI pour créer le projet Symfony et lancer le serveur en local ;
- Node.js et npm pour la gestion des paquets et le projet React ;
- Docker et Docker Compose pour lancer l'API, la bdd MySQL et le front dans des conteneurs ;
- Git et GitHub pour le versioning ;
- Bruno pour tester les routes de l'API.

Bruno pour tester l'API :

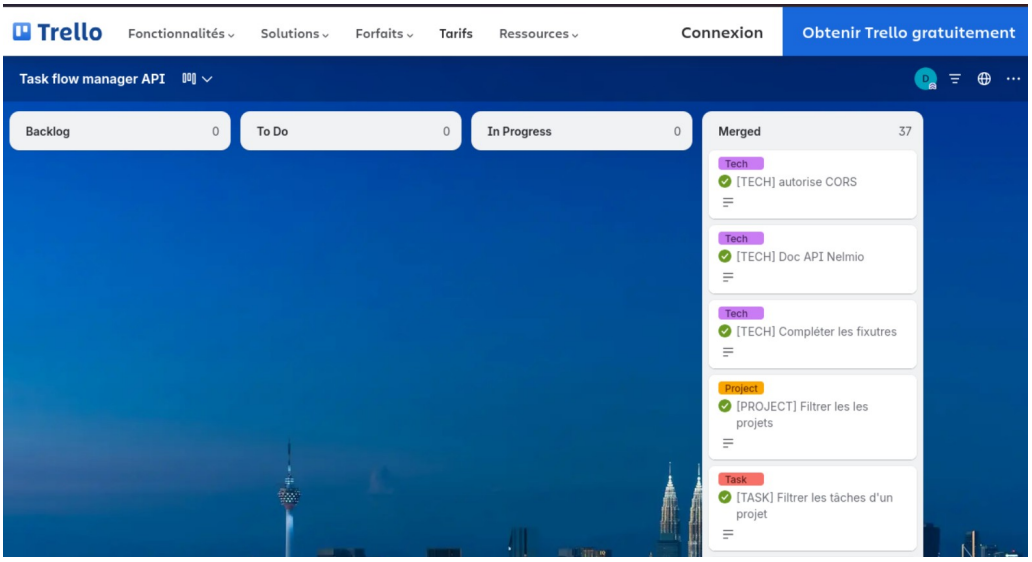
Pour tester les routes de l'API sans passer par le front, j'ai utilisé Bruno. J'ai créé une collection de requêtes avec un environnement qui contient l'URL de l'API et le token JWT. Le token est récupéré avec la route de login, puis réutilisé automatiquement dans les autres requêtes.



Requête de login dans Bruno renvoyant le token JWT

Trello pour organiser le travail :

Pour organiser les fonctionnalités à développer, j'ai créé un tableau Trello avec plusieurs colonnes (Backlog, To Do, In Progress, Merged). Chaque fonctionnalité est une carte, et chaque carte correspond à une branche Git. Je vois ainsi où j'en suis et je garde le lien entre ce qui est prévu et le code.



Vue d'ensemble du tableau Trello

Initialisation du projet API Symfony :

J'ai créé le projet en Symfony 7.2, avec le squelette sans la partie webapp, puisque le front est un projet séparé. J'ai ensuite installé les bundles nécessaires à l'API :

- Doctrine ORM et Doctrine Migrations : accès à la base de données ;
- LexikJWTAuthenticationBundle : authentification par token JWT ;
- NelmioCorsBundle : autoriser le front React à appeler l'API ;
- NelmioApiDocBundle : documentation Swagger de l'API, accessible sur /api/doc ;
- Symfony Security et Symfony Validator : sécurité et validation des données reçues ;
- DoctrineFixturesBundle : jeux de données pour le développement.

Les variables d'environnement et la sécurité des secrets :

J'ai fait particulièrement attention à ne jamais mettre de secret dans Git. J'ai organisé les fichiers .env comme ceci :

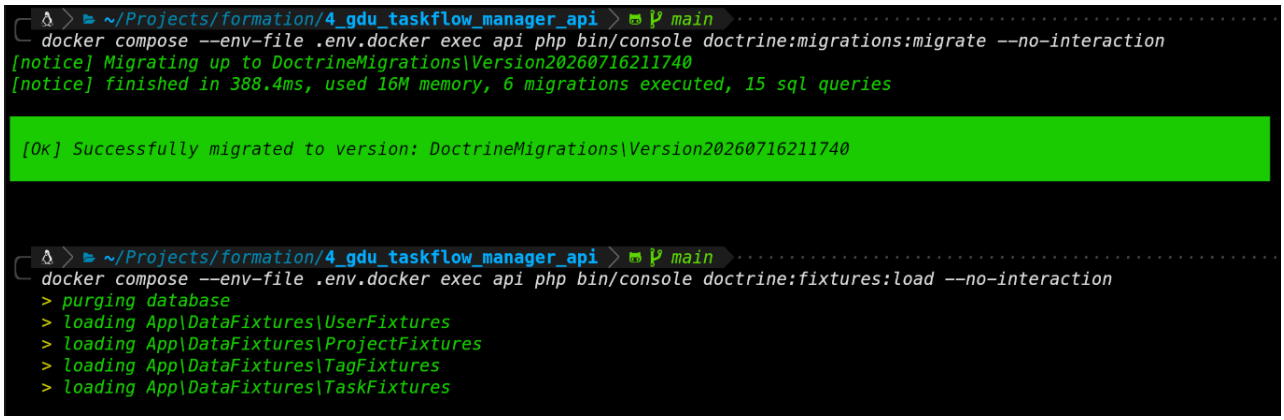
Fichier	Versionné	Rôle
.env	Oui	Modèle avec toutes les variables, sans valeur
.env.local	Non	Mes vraies valeurs pour travailler en local
.env.docker	Non	Les valeurs utilisées par Docker Compose
.env.test	Oui	Configuration des tests (bdd SQLite en mémoire)

Les variables principales sont **APP_SECRET**, **DATABASE_URL**, les identifiants MySQL, **JWT_PASSPHRASE** et **CORS_ALLOW_ORIGIN**.

Pour le JWT, j'ai généré la paire de clés (privée et publique) avec la commande php bin/console lexik:jwt:generate-keypair. Les clés sont stockées dans config/jwt et ne sont pas versionnés. La clé privée est également protégée par la passphrase JWT_PASSPHRASE.

La base de donnée MySQL 8 :

J'ai choisi MySQL 8 comme base de données relationnelle. Le schéma est généré avec les migrations Doctrine, ce qui permet de suivre l'historique des modifications de la base. Pour avoir des données de test, j'ai créé des fixtures chargées avec DoctrineFixturesBundle. Pour les tests automatisés, j'utilise SQLite en mémoire, ce qui est plus rapide et ne touche pas à la bdd de développement.

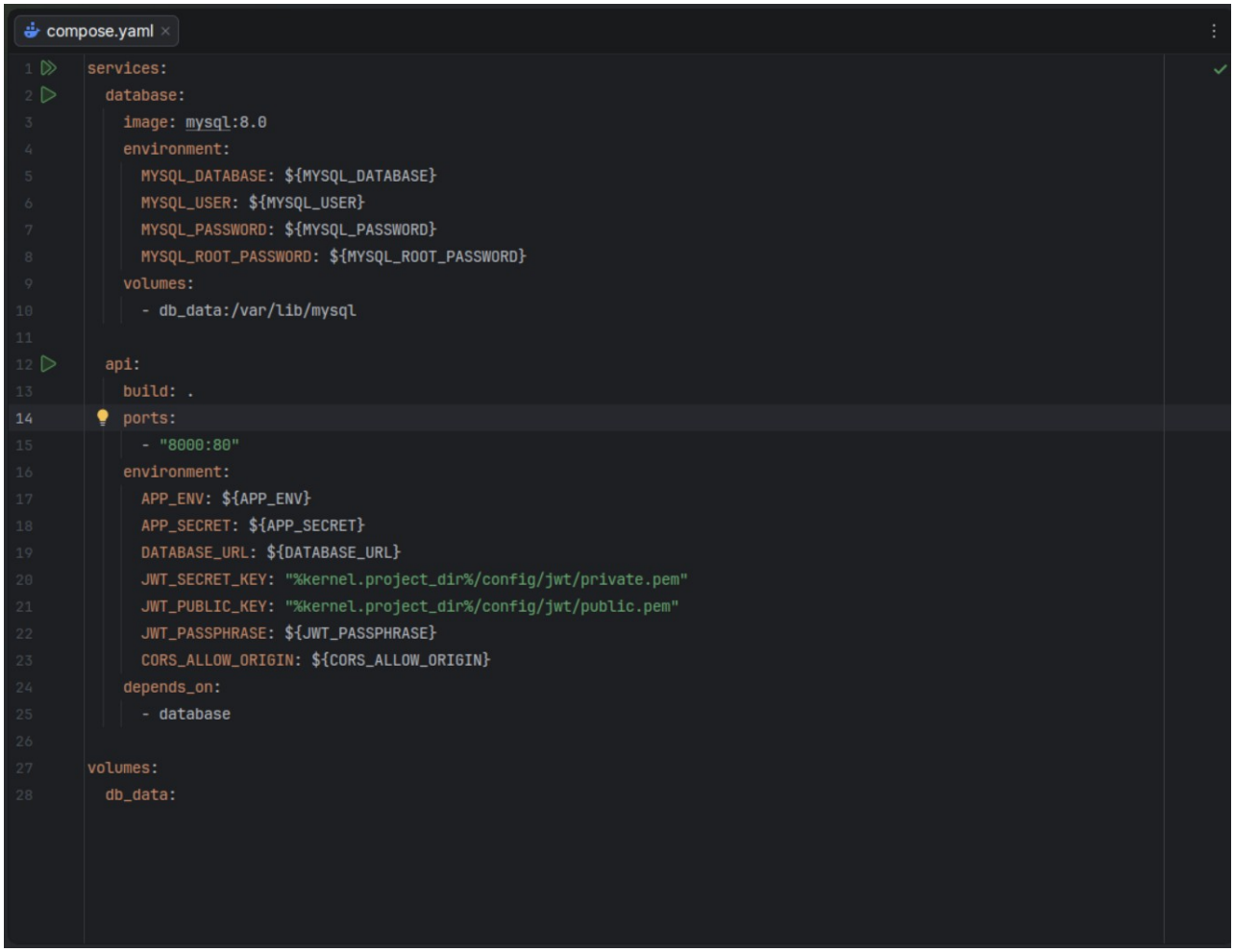


Terminal avec migrations + fixtures

Docker :

Afin que les projets se lancent de la même façon partout, j'ai utilisé Docker pour les conteneuriser :

- Dockerfile basé sur l'image php:8.2-apache, qui installe les extensions PHP (pdo_mysql, zip), Composer, les dépendances du projet, et génère les clés JWT ;
- compose.yaml avec deux services : api (exposé sur le port 8000) et database (MySQL 8, avec un volume pour conserver les données) ;
- compose.override.yaml, utilisé uniquement en local, qui expose MySQL sur le port 3307 pour que je puisse m'y connecter depuis mon éditeur.



Fichier compose.yaml



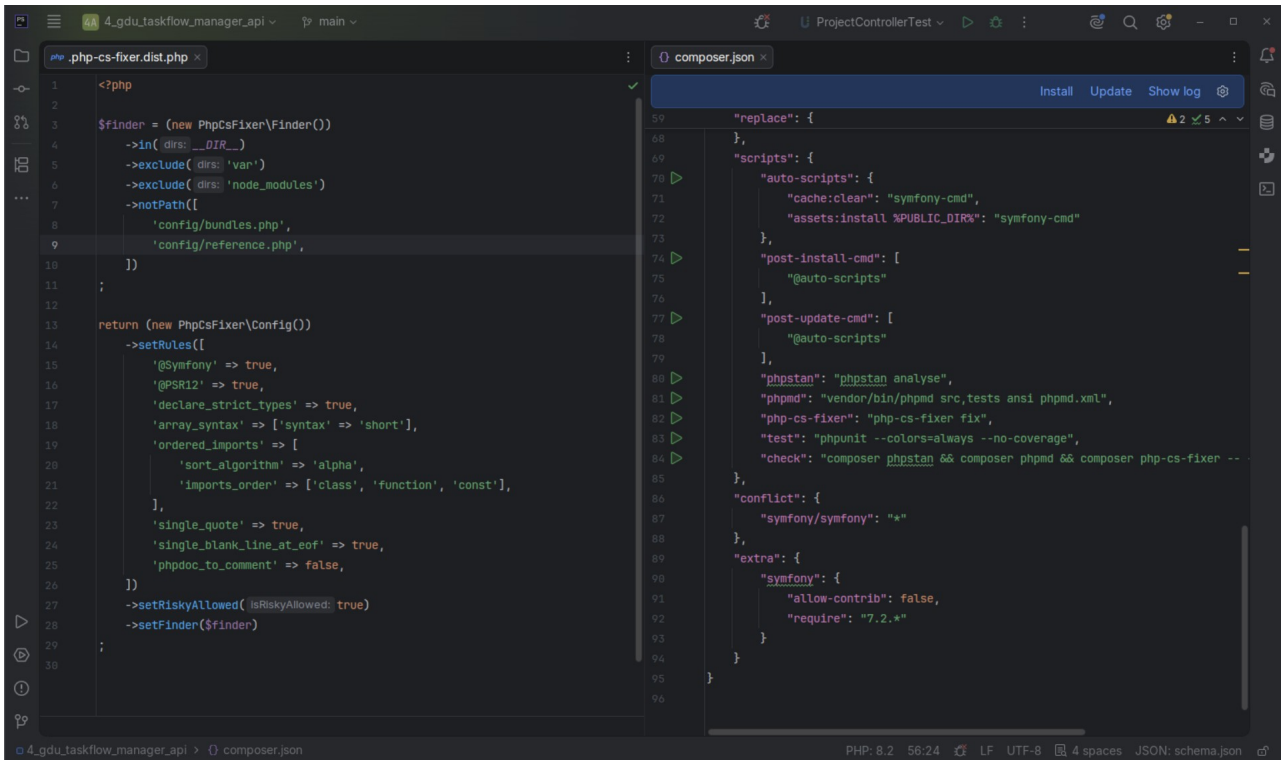
***docker compose ps -a** avec tous les conteneurs **api** et **bdd** démarrés*

Les outils de qualité de code :

Dès le début des projets, j'ai mis en place des outils afin de garder le code le plus propre possible et éviter un maximum de bugs :

- PHP CS Fixer qui formate le code automatiquement selon les standards **Symfony** et **PSR-12**. J'y ai également ajouté quelques règles comme le tri des uses, les singles quotes etc ;
- PHPStan qui fait une analyse statique du code, afin de trouver des erreurs ;
- PHPMD qui détecte le code mort, les méthodes trop longues etc ;
- PHPUnit 11 qui exécute les tests fonctionnels des contrôleurs.

J'ai regroupé tout ça dans des scripts Composer, ce qui me permet de lancer toutes ces vérifications facilement. La commande **composer check**, par exemple, lance PHPStan, PHPMD, PHP CS Fixer ainsi que les suites de tests.



Ensemble de règles et scripts

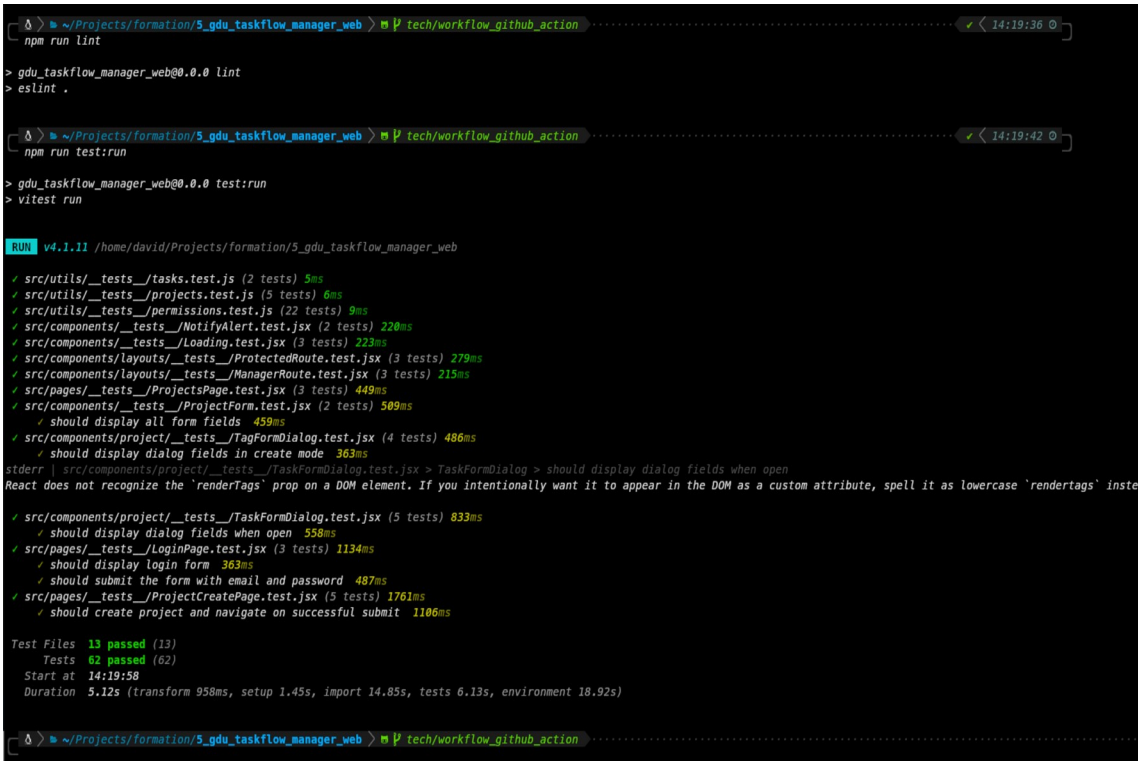
Initialisation du projet Web React :

Afin de créer le front avec Vite et React 19, j'ai installé différents éléments :

- Material UI (MUI) pour les composants graphiques ;
- React Router pour la navigation entre les différentes pages ;
- ESLint pour la qualité du code JavaScript ;
- Vitest + Testing Library + jsdom pour les tests des composants.

L'url de l'API est dans une variable d'environnement **VITE_API_BASE_URL**. Ce n'est pas un secret, Vite l'intègre au moment du build. Côté API j'ai configuré le CORS (nelmio) pour autoriser le front et le header Authorization.

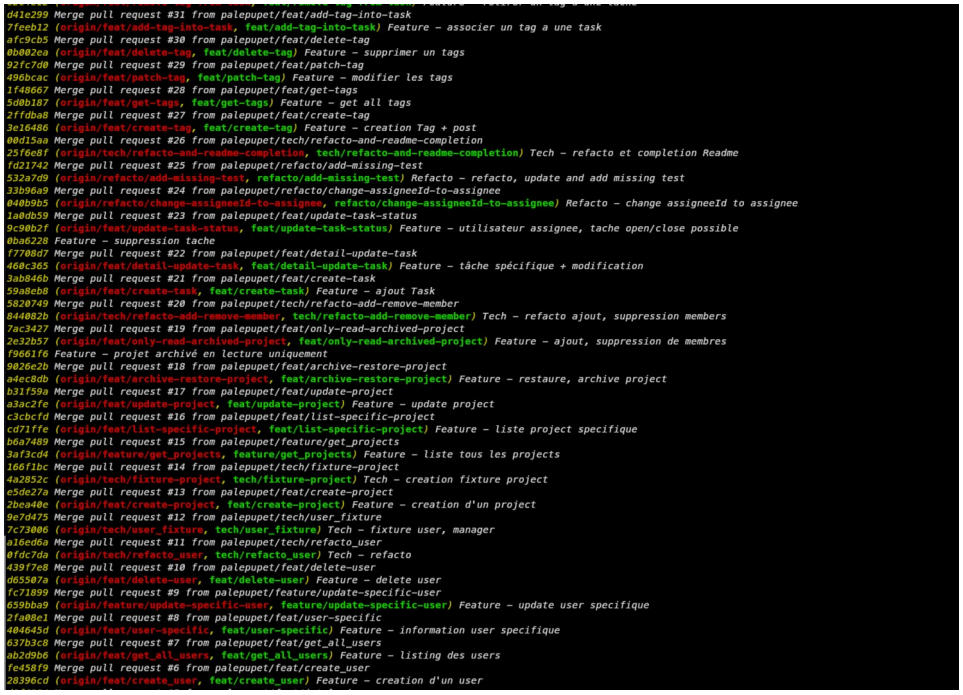
Le front a également son Dockerfile, une étape Node pour builder, puis nginx pour les fichiers statiques.



Lint ESLint et tests Vitest du front

Git et GitHub :

J'ai créé un dépôt GitHub pour chaque projet. Je travaille avec une branche par fonctionnalité, nommée par type (**feat/..., fix/..., tech/...**). Chaque branche est ensuite fusionnée dans la branche **main** par **pull request**. Ce qui donne un historique propre et clair.



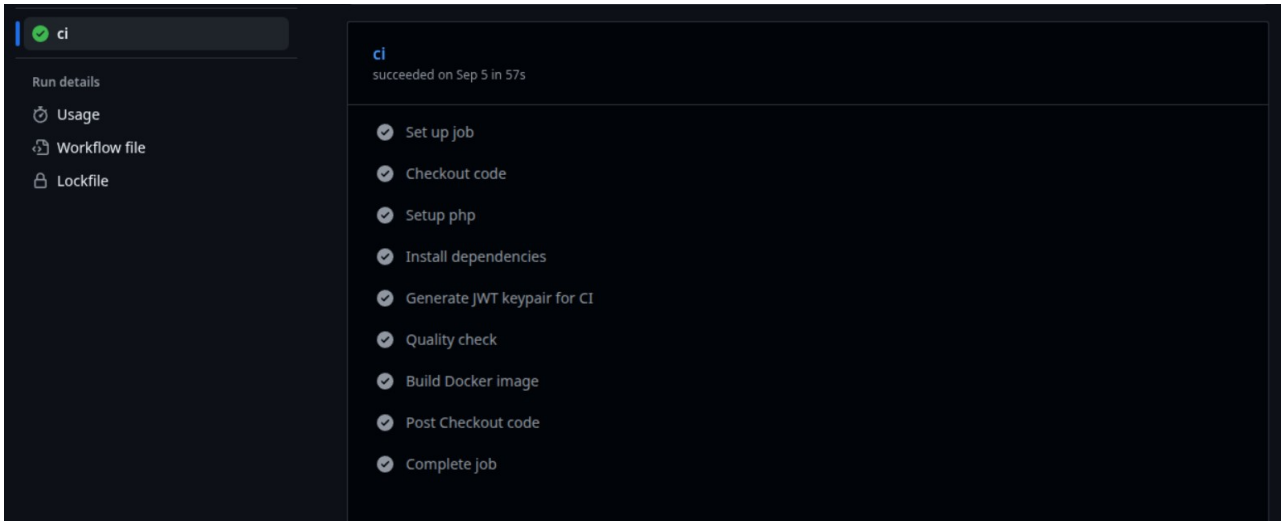
*Historique des merges (**git log --oneline**)*

Intégration continue (CI) avec GitHub Actions :

Pour ne pas dépendre uniquement de mes vérifications en local, j'ai mis en place une intégration continue avec GitHub Actions sur les deux projets (api + front). Le fichier **.github/workflows/ci.yml** est lancé automatiquement à chaque push sur toutes les branches. Je vois donc tout de suite si une modification casse quelque chose avant de fusionner ma pull request dans ma branche main.

Côté API, la CI installe PHP 8.2 et les dépendances, génère les clés JWT pour les tests, lance composer check (PHPStan, PHPMD, PHP CS Fixer et PHPUnit) puis vérifie que l'image Docker se construit correctement.

Côté Web, la CI installe Node 22 et les dépendances (npm ci), lance ESLint et les tests Vitest, fait le build de production avec Vite, puis le build de l'image Docker.



***CI** valide avant fusion des branches*

Si une étape échoue, le pipeline passe au rouge et je corrige immédiatement avant de fusionner la branche. Ce qui me garantit que le code sur la branche main respecte toujours les règles de qualité et que les tests passent.

2. Précisez les moyens utilisés :

Environnement matériel :

- Modèle : Lenovo IdeaPad Slim 5 ;
- Système : Linux - Zorin OS 18.1 ;
- RAM : 16 Go.

Logiciels et outils :

- PhpStorm + plugin ;
- PHP 8.2, Composer 2, Symfony CLI ;
- Symfony 7.2, Doctrine ORM, LexikJWT, NelmioCors ;
- MySQL 8 (dev), SQLite (test)
- Docker, Docker Compose ;
- PHP CS Fixer, PHPStan, PHPMD, PHPUnit ;
- Node.js, npm, Vite, Vitest, React 19, MUI, ESLint ;
- Git, GitHub, Bruno, Trello.

3. Avec qui avez-vous travaillé ?

J'ai mis en place cet environnement seul, en m'appuyant sur les consignes de la formation et sur la documentation officielle (Symfony, Doctrine, LexikJWT, Docker, Vite…).

J'ai pu échanger avec mon formateur sur les bonnes pratiques, par exemple la gestion des secrets et de la CI pendant les sessions de formation à distance.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager : Initialisation, configuration et mise en place de l'API Symfony + Front React**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

- Dépôt GitHub de l'API : https://github.com/palepupet/gdu_taskflow_manager_api
- Dépôt GitHub du front : https://github.com/palepupet/gdu_taskflow_manager_web
- Application en ligne : https://gdu-taskflow-manager-web.onrender.com
- Documentation de l'API (Swagger) : https://gdu-taskflow-manager-api.onrender.com/api/doc

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 2 Analyser les besoins et maquetter les interfaces utilisateur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

La société TaskFlow gérait ses projets et ses tâches avec une API, utilisable uniquement par requêtes HTTP. Ce n'était pas pratique au quotidien et ce n'était pas accessible à tous les collaborateurs. L'entreprise a donc demandé une interface web pour gérer les projets plus facilement.

Avant de développer, j'ai analysé le besoin pour savoir qui utilise l'application, ce que chacun peut faire, et à quoi ressembleront les écrans.

Étape 1 : Analyse du besoin

À partir du besoin métier, j'ai listé les fonctionnalités attendues :

- se connecter et se déconnecter ;
- consulter, créer, modifier, archiver et restaurer des projets ;
- gérer les membres d'un projet ;
- gérer les tâches d'un projet : création, modification, suppression, assignation, changement d'état, priorité, échéance ;
- filtrer et trier les tâches ;
- gérer des tags par projet et les associer aux tâches ;
- modifier son profil ;
- administrer les utilisateurs (réservé aux managers).

J'ai aussi relevé les contraintes :

- l'application doit être sécurisée : chaque utilisateur ne voit et ne modifie que ce qui le concerne ;
- un projet archivé n'est plus modifiable, mais reste consultable ;
- l'interface consomme uniquement l'API et n'a pas de base de données propre.

Étape 2 : Identification des acteurs et de leurs rôles

J'ai identifié les différents profils d'utilisateurs. Un même utilisateur peut avoir plusieurs rôles selon le projet :

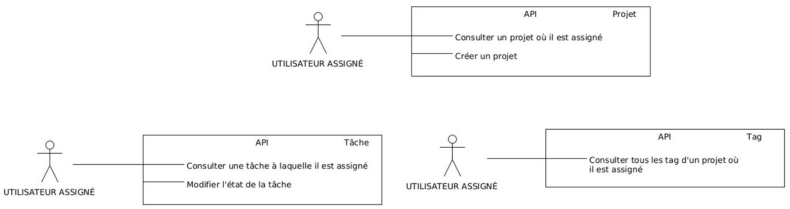
- Manager : administre les utilisateurs et a tous les droits sur les projets ;
- Owner : créateur d'un projet, il le gère (modification, archivage, membres, tâches, tags) ;

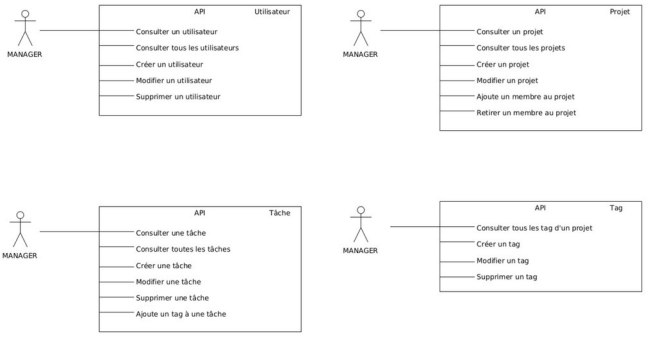
- Membre : consulte le projet auquel il participe, ses tâches et ses tags ;
- Utilisateur assigné : peut faire évoluer l'état de la tâche qui lui est assignée ;
- Utilisateur : utilisateur connecté, qui peut créer ses propres projets ;
- Utilisateur non connecté : personne qui n'a aucun droit sur le projet.

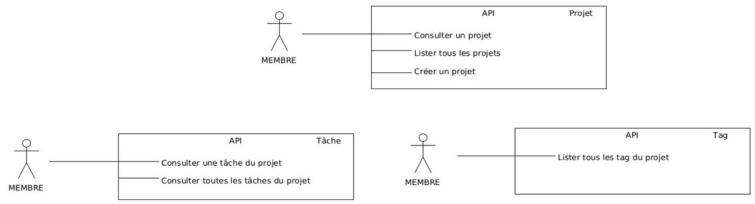
Étape 3 : Diagrammes de cas d'utilisation

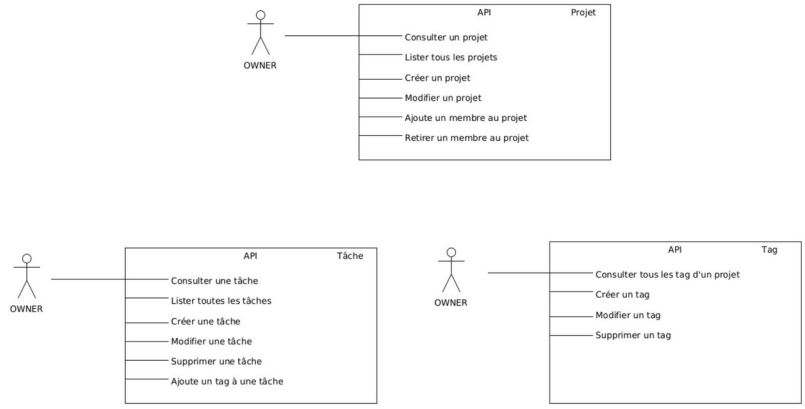
Pour chaque rôle, j'ai réalisé un diagramme de cas d'utilisation. Il représente les actions possibles sur chaque ressource : utilisateurs, projets, tâches et tags.

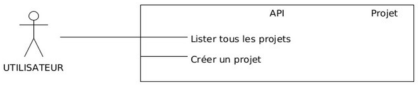
Ci-dessous les différents diagrammes de cas d'utilisation :











Ces diagrammes m'ont permis de vérifier que chaque fonctionnalité du besoin correspondait bien à au moins un acteur, et de préparer la liste des routes de l'API.

Étape 4 : Matrice d'autorisation

Pour aller plus loin que les cas d'utilisation, j'ai construit une matrice de droits. Elle croise chaque action (en lignes) avec chaque rôle (en colonnes) : vert = autorisé, rouge = refusé.

Elle est organisée par ressource : Auth, Utilisateurs, Projets en cours, Projets archivés, Tâches, Tags.

Action	MANAGER	OWNER	MEMBRE	ASSIGNÉ TÂCHE	UTILISATEUR	HORS PROJET
Auth						
POST /auth/login	✓	✓	✓	✓	✓	✓
GET /me	✓	✓	✓	✓	✓	✗
PATCH /me	✓	✓	✓	✓	✓	✗
Utilisateurs						
Lister / voir / créer / modifier /supprimer un user	✓	✗	✗	✗	✗	✗
Projets (en cours)						
Créer un projet	✓	✓	✓	✓	✓	✗
Lister les projets	✓	✓	✓	✗	✓	✗
Voir un projet spécifique	✓	✓	✓	✓	✗	✗
Modifier / archiver / restaurer	✓	✓	✗	✗	✗	✗
Ajouter / retirer des membres	✓	✓	✗	✗	✗	✗
Projets (archivés)						
Lecture (projet, tâches, tags)	✓	✓	✓	✓	✗	✗
Écriture	✗	✗	✗	✗	✗	✗
Tâches						
Lister / voir celles du projet	✓	✓	✓	✓	✗	✗
Créer une tâche	✓	✓	✗	✗	✗	✗
Modifier : titre, description, Priorité, échéance, assignee	✓	✓	✗	✗	✗	✗
Supprimer une tâche	✓	✓	✗	✗	✗	✗
Passer l'état à en cours	✓	✓	✗	✗	✗	✗
Passer l'état à ouvert / terminé	✓	✓	✗	✓	✗	✗
Associer un tag à une tâche	✓	✓	✗	✗	✗	✗
Tags						
Lister les tags	✓	✓	✓	✓	✗	✗
Créer / renommer / supprimer un tag	✓	✓	✗	✗	✗	✗

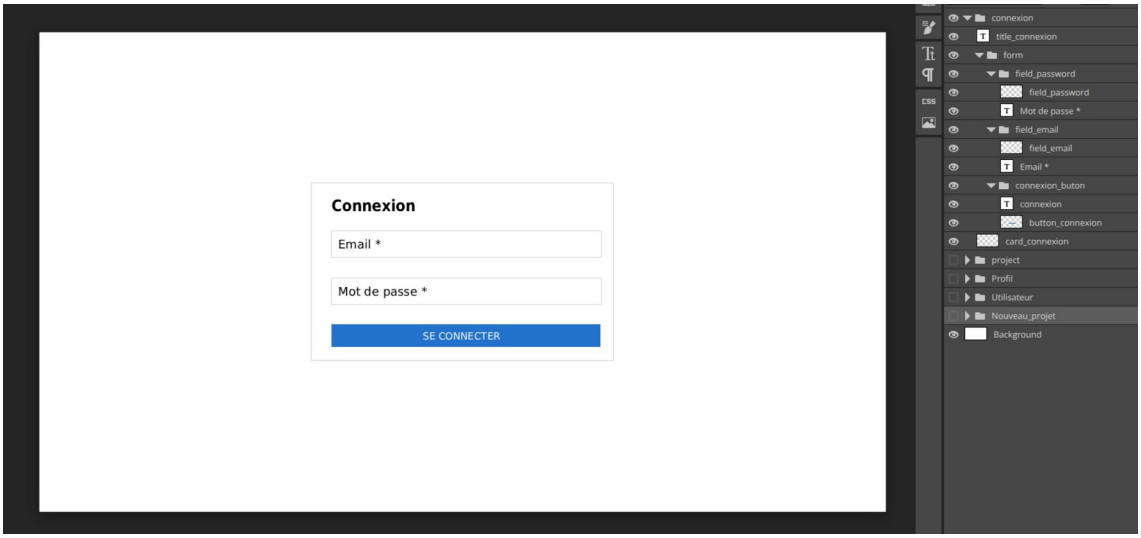
Matrice d'autorisation par rôle

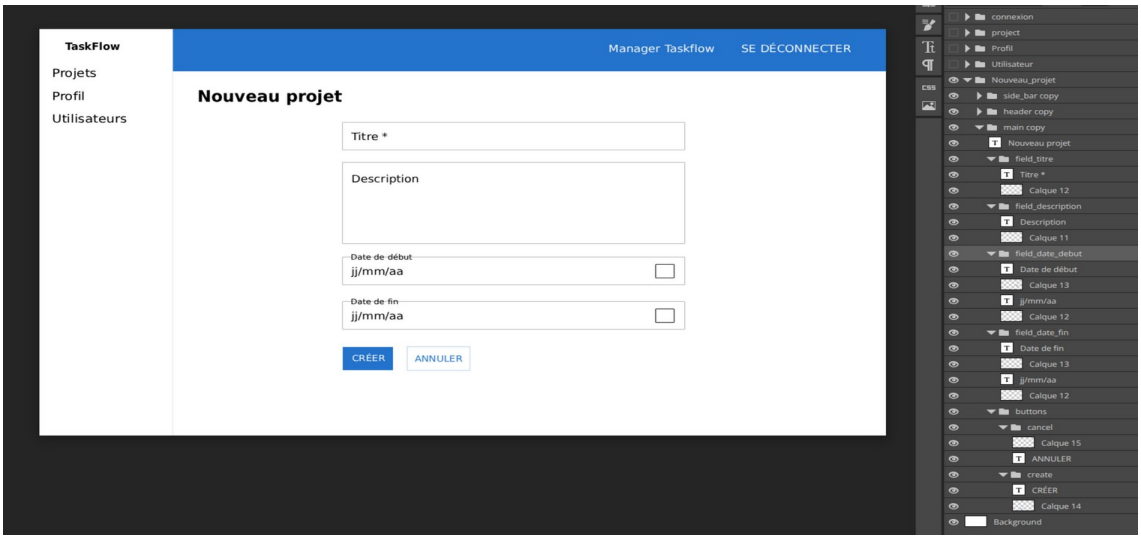
Cette matrice m'a servi de référence pendant tout le développement : pour coder les contrôles d'accès de l'API, pour afficher ou masquer les boutons dans le front, et pour écrire les tests de sécurité.

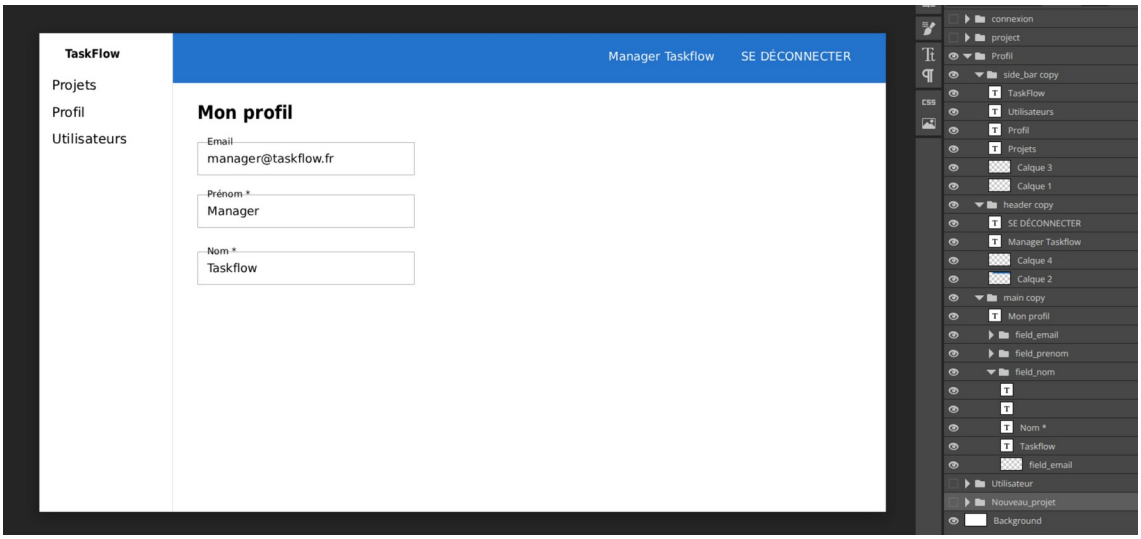
Étape 5 : Maquettage des interfaces

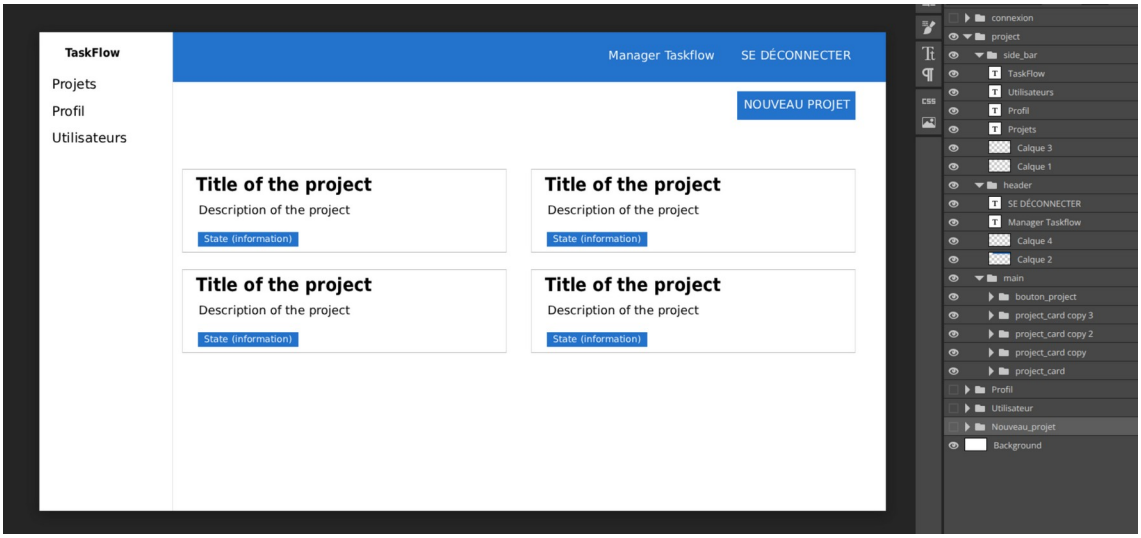
J'ai ensuite réalisé les maquettes des principaux écrans avec Photopea. J'ai organisé les calques par écran et par zone (sidebar, header, main), pour retrouver et réutiliser facilement les éléments.

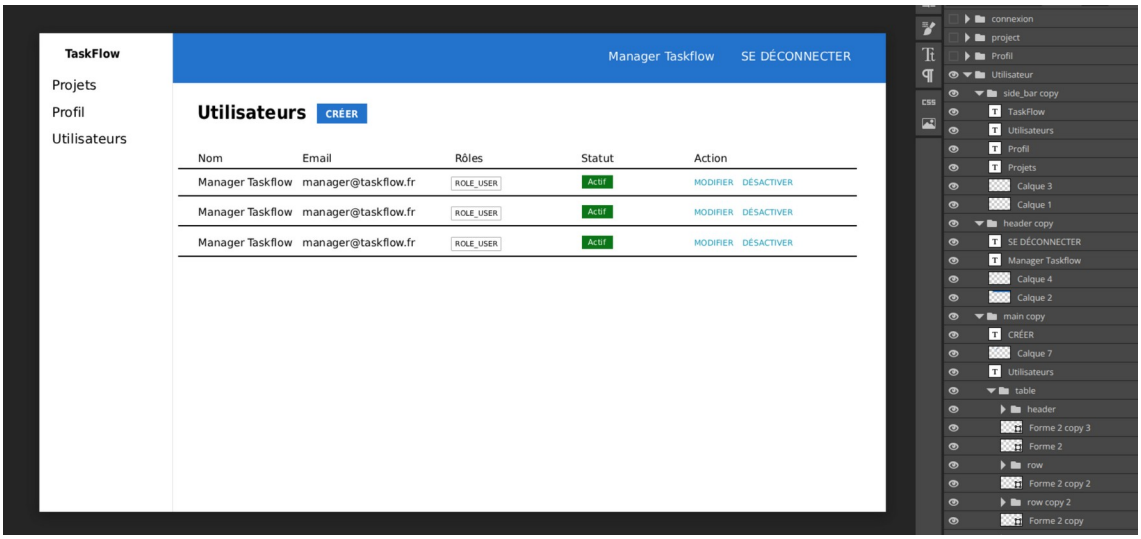
Ci-dessous les différentes maquettes :


Connexion
Email *
Mot de passe *
SE CONNECTER
connexion
title_connexion
form
field_password
field_password
Mot de passe *
field_email
field_email
Email *
connexion_buton
connexion
button_connexion
card_connexion
project
Profil
Utilisateur
Nouveau_projet
Background


TaskFlow
Projets
Profil
Utilisateurs
Manager Taskflow
SE DÉCONNECTER
Nouveau projet
Titre *
Description
Date de début
jj/mm/aa
Date de fin
jj/mm/aa
CRÉER
ANNULER
connexion
project
Profil
Utilisateur
Nouveau_projet
side_bar copy
header copy
main copy
Nouveau projet
field_titre
Titre *
Calque 12
field_description
Description
Calque 11
field_date_debut
Date de début
Calque 13
jj/mm/aa
Calque 12
field_date_fin
Date de fin
Calque 13
jj/mm/aa
Calque 12
buttons
cancel
Calque 15
ANNULER
create
CRÉER
Calque 14
Background


TaskFlow
Projets
Profil
Utilisateurs
Manager Taskflow
SE DÉCONNECTER
Mon profil
Email
manager@taskflow.fr
Prénom *
Manager
Nom *
Taskflow


TaskFlow
Projets
Profil
Utilisateurs
Manager Taskflow
SE DÉCONNECTER
NOUVEAU PROJET
Title of the project
Description of the project
State (information)
Title of the project
Description of the project
State (information)
Title of the project
Description of the project
State (information)
Title of the project
Description of the project
State (information)



J'ai défini une mise en page commune à tous les écrans connectés : une barre latérale de navigation (Projets, Profil, Utilisateurs), un bandeau supérieur avec le nom de l'utilisateur et le bouton de déconnexion, et une zone principale pour le contenu.

Étape 6 : Découpage en fonctionnalités
Enfin, j'ai découpé les fonctionnalités en cartes dans Trello. Chaque carte correspond à une fonctionnalité, puis à une branche Git.

2. Précisez les moyens utilisés :

- Cahier des charges du projet ;
- UMLet pour créer les diagrammes ;
- Excel pour la matrice des droits ;
- Photopea pour les maquettes ;
- Trello pour la gestion des use cases.

3. Avec qui avez-vous travaillé ?

J'ai réalisé l'analyse et les maquettes seul, à partir du cahier des charges fourni dans le cadre de la formation. J'ai présenté mes diagrammes et ma matrice d'autorisation à mon formateur, qui les a validés avant le début du développement.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager – Analyse des besoins, rôles et droits d'accès, maquettage de l'interface web**
Période d'exercice	**Du** 29/06/2026 **au** 05/09/2026

5. Informations complémentaires *(facultatif)*

Tableau Trello : https://trello.com/b/KJUtMTTP/task-flow-manager-api

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 3 Définir l'architecture logicielle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'application web TaskFlow Manager est un front React séparé de l'API Symfony. Elle n'a pas de base de données : toutes les données passent par l'API, en JSON, avec une authentification par token JWT. Avant de coder les écrans, j'ai défini comment organiser le projet. Je voulais que chaque partie ait un rôle clair et que le code soit facile à faire évoluer et à tester.

L'application repose sur deux projets indépendants :

- le front (React) gère l'affichage et l'interaction avec l'utilisateur ;
- l'API (Symfony) gère les données, les règles métier et la sécurité.

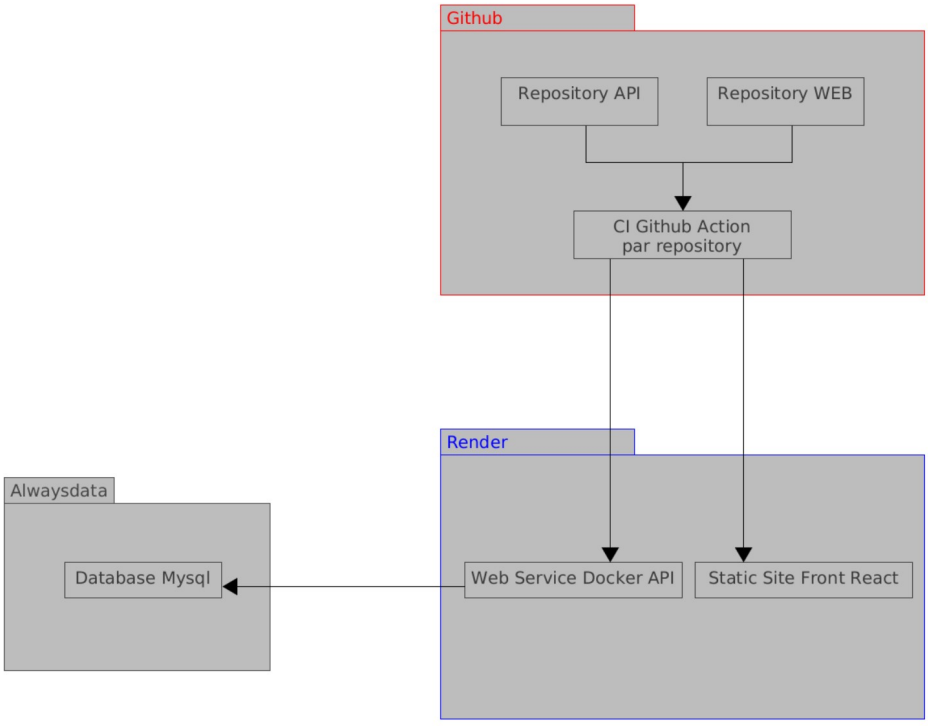


Schéma d'architecture de déploiement

Architecture en couches :

J'ai découpé le dossier src/ en plusieurs couches, chacune avec un rôle précis :

Dossier	Rôle
api/	Les appels à l'API : un fichier par ressource (auth, projects…) et un client commun client.js
contexts/ + hooks/	L'état global de l'authentification (AuthProvider, useAuth)
pages/	Une page par écran (Connexion, Projects…)
components/	Les composants réutilisables : mise en page (layouts/), éléments du détail d'un projet, formulaires, …
utils/	La logique pure : les permissions et les fonctions
theme/	Le thème Material UI

Les composants n'appellent jamais fetch directement. Ils passent par les fonctions de api/, ce qui sépare l'affichage de l'accès aux données. Si une route de l'API change, je n'ai qu'un fichier à modifier.

Un client unique :

J'ai écrit un seul point d'entrée pour toutes les requêtes : la fonction **apiClient** dans **api/client.js**.

- Elle construit l'URL à partir de la variable d'environnement **VITE_API_BASE_URL** ;
- Elle ajoute automatiquement le token **JWT** stocké en **sessionStorage** ;
- Elle convertit le corps de la requête en **JSON** ;
- Elle gère les erreurs.

Authentification :

L'état de connexion est partagé dans toute l'application grâce au Context API de React (**AuthProvider**). Au démarrage, s'il y a un token, l'application appelle **GET /me** pour récupérer l'utilisateur connecté. Les composants accèdent à l'utilisateur et aux fonctions **login/logout** avec le **hook useAuth()**.

J'ai choisi le **Context** plutôt qu'une bibliothèque comme **Redux** car l'état global se limite à l'utilisateur connecté uniquement.

Le routage :

La navigation est gérée avec **React Router**. Les routes sont imbriquées dans **App.jsx** :

- **/login** est publique ;
- toutes les autres pages sont protégées par **ProtectedRoute**, qui redirige vers **/login** si l'utilisateur n'est pas connecté ;
- **/users** est en plus protégée par **ManagerRoute**, qui est uniquement accessible aux managers ;
- les pages connectées partagent le même **DashboardLayout** (**Navbar**, **Sidebar**, **contenu**), ce qui reprend la mise en page des maquettes.

```
function App() {
  return (
    <Routes>
        <Route path="/login" element={<LoginPage />} />

        <Route element={<ProtectedRoute />}>
            <Route element={<DashboardLayout />}>
                <Route path="/projects" element={<ProjectsPage />} />
                <Route path="/projects/create" element={<ProjectCreatePage />} />
                <Route path="/projects/:id" element={<ProjectDetailPage />} />
                <Route path="/projects/:id/edit" element={<ProjectEditPage />} />
                <Route path="/profile" element={<ProfilePage />} />
                <Route element={<ManagerRoute />}>
                    <Route path="/users" element={<UsersPage />} />
                </Route>
            </Route>
        </Route>

        <Route path="*" element={<Navigate to="/projects" />} />
    </Routes>
  )
}
```

Protection des routes

Les permissions :

J'ai regroupé toutes les règles d'affichage dans **utils/permissions.js**, sous forme de fonctions simples : **isManager**, **isOwner**, **canEditProject** etc. Elles reprennent la matrice d'autorisation et servent à afficher ou masquer les différents éléments selon le rôle.

La vraie sécurité est assurée par l'API, qui revérifie les droits à chaque requête. Même si quelqu'un contourne l'interface, il ne peut pas faire une action interdite.

```php
export function canEditProject(user, project) {
    if (!user || !project) {
        return false;
    }

    if (project.isArchived) {
        return false;
    }

    return isManager(user) || isOwner(user, project);
}

Show usages  David
export function canManageProject(user, project) {
    if (!user || !project) {
        return false;
    }

    return isManager(user) || isOwner(user, project);
}

Show usages  David
export function canArchiveProject(user, project) {
    return canManageProject(user, project) && !project.isArchived;
}
```

*Extrait des règles de **permissions.js***

2. Précisez les moyens utilisés :

- React 19 et Vite : bibliothèque et outil de build ;
- React Router 7 : routage et routes protégées ;
- Context API et hooks React : état d'authentification ;
- Material UI : composants et thème ;
- fetch natif : via un client centralisé ;
- Docker et nginx : service de l'application buildé ;
- Documentation officielle de React, React Router et Vite.

3. Avec qui avez-vous travaillé ?

J'ai défini cette architecture seul, en m'appuyant sur les consignes de la formation et sur la documentation officielle de React et de React Router.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Architecture de l'application web React**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

- Dépôt GitHub du front : https://github.com/palepupet/gdu_taskflow_manager_web
- Application en ligne : https://gdu-taskflow-manager-web.onrender.com

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 4 Développer les interfaces utilisateur

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

À partir des maquettes et de l'architecture définie, j'ai développé les écrans de l'application web TaskFlow Manager en React 19 avec Material UI. Je voulais une interface simple et cohérente sur tous les écrans, qui s'adapte aux droits de l'utilisateur connecté.

Mise en page commune :

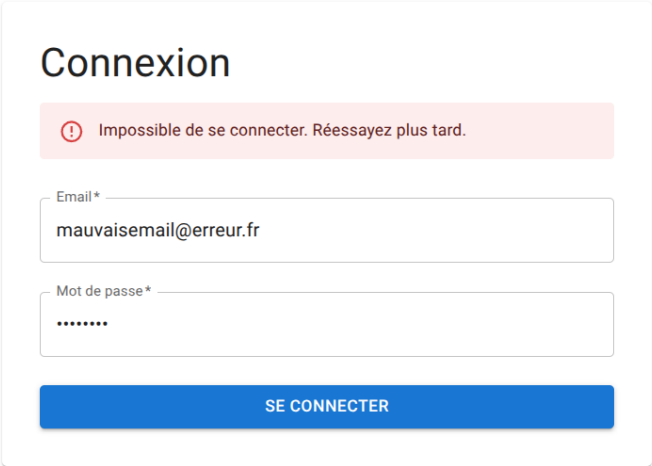
Toutes les pages connectées partagent le même **DashboardLayout** :

- une **Navbar** en haut, avec le nom de l'utilisateur et le bouton de déconnexion ;
- une **Sidebar** à gauche, avec les liens Projets, Profil et Utilisateurs. Le lien actif est mis en surbrillance, et le lien Utilisateurs n'apparaît que pour un manager ;
- une zone principale qui affiche la page en cours (**<Outlet />** de React Router).

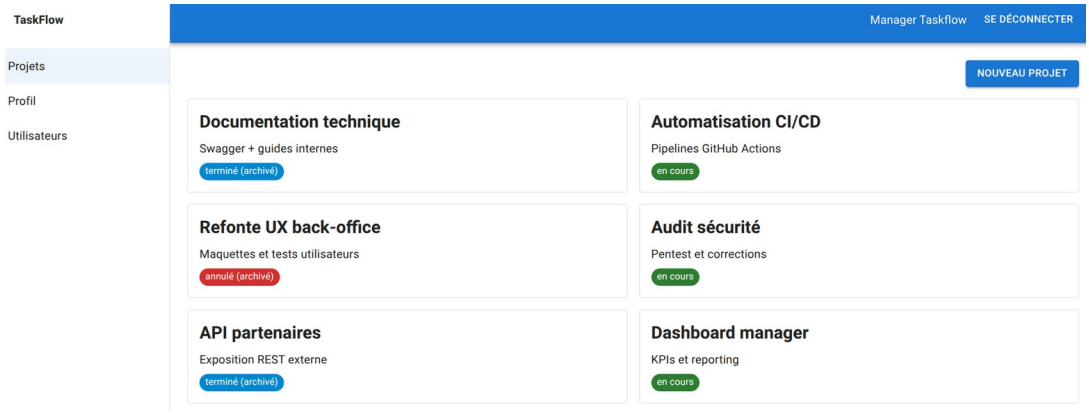
J'ai défini les couleurs une seule fois dans un thème MUI (**muiTheme.js**), pour que tous les composants aient le même style.

Les écrans :

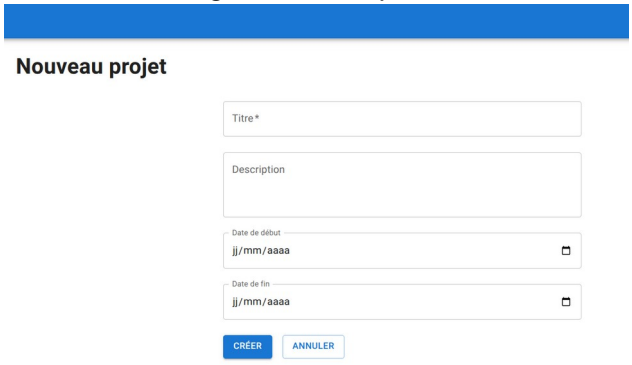
- **Connexion :** formulaire email et mot de passe. Les erreurs (identifiants incorrects, session expirée) s'affichent dans une alerte, et le bouton est désactivé pendant l'envoi.

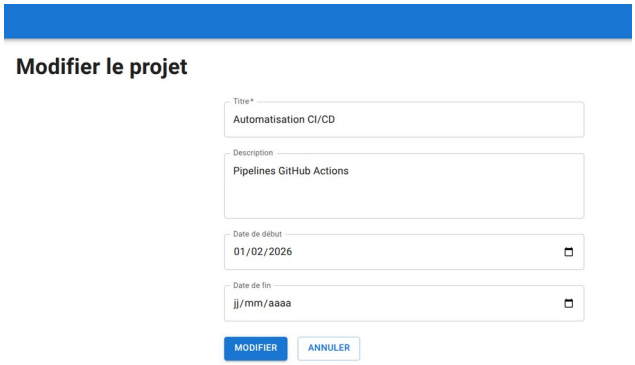


- **Liste des projets** : les projets s'affichent sous forme de cartes cliquables (titre, description, état sous forme de badge), avec un bouton **Nouveau projet**.



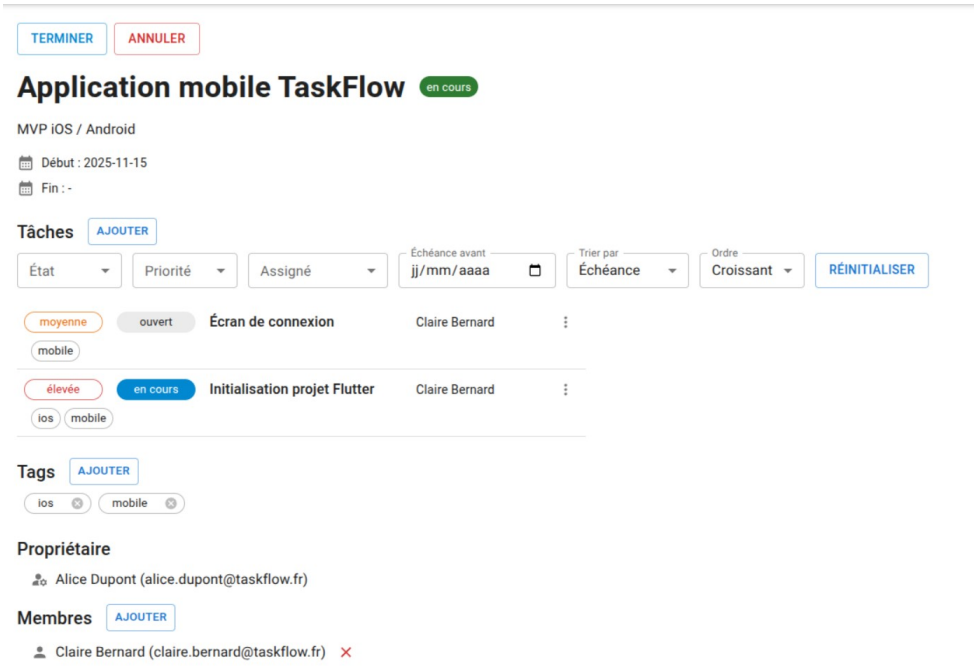
- **Création et modification d'un projet :** un seul composant ProjectForm, réutilisé par les deux pages (titre obligatoire, description, dates de début et de fin).

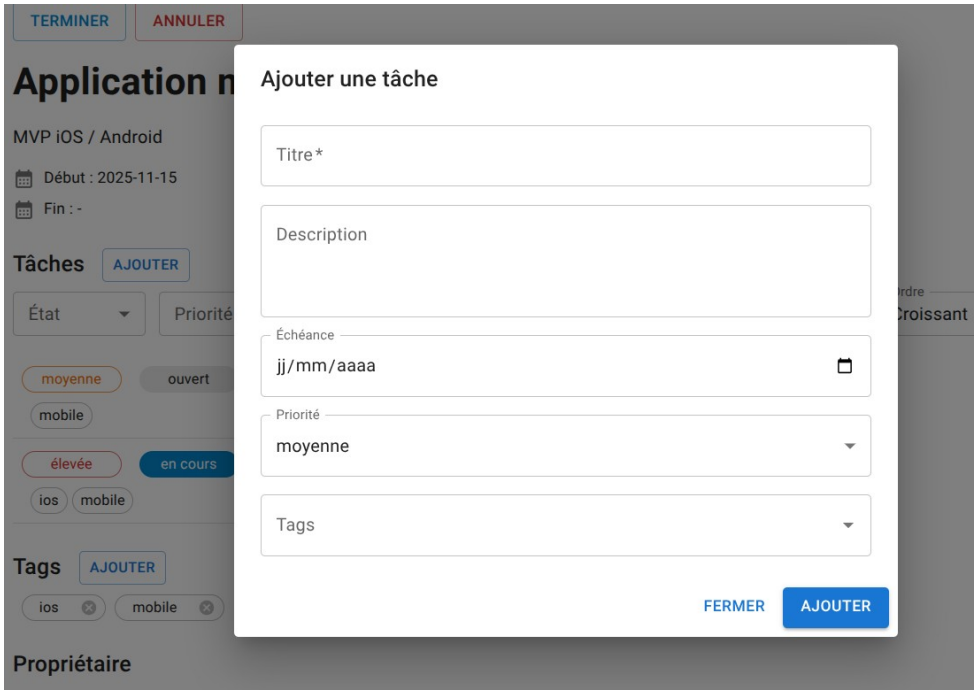




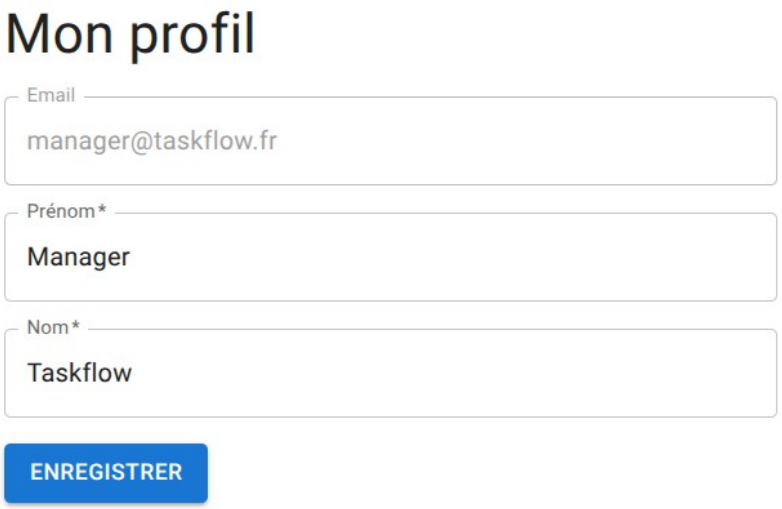
- **Détail d'un projet :** c'est l'écran principal de l'application. Il regroupe :
  - les informations du projet et ses actions (modifier, archiver, restaurer) ;
  - la liste des tâches, avec des badges de couleur pour la priorité et l'état ;
  - une barre de filtres et de tri (état, priorité, assigné, échéance, tri croissant ou décroissant) ;
  - la gestion des tags et des membres.

Les créations et modifications se font dans des boîtes de dialogue (**TaskFormDialog**, **TagFormDialog**, **AddMembersDialog**), pour rester sur la page. Pour choisir les tags d'une tâche ou ajouter des membres, j'ai utilisé un champ **Autocomplete** à sélection multiple. Un bandeau signale quand un projet est archivé.

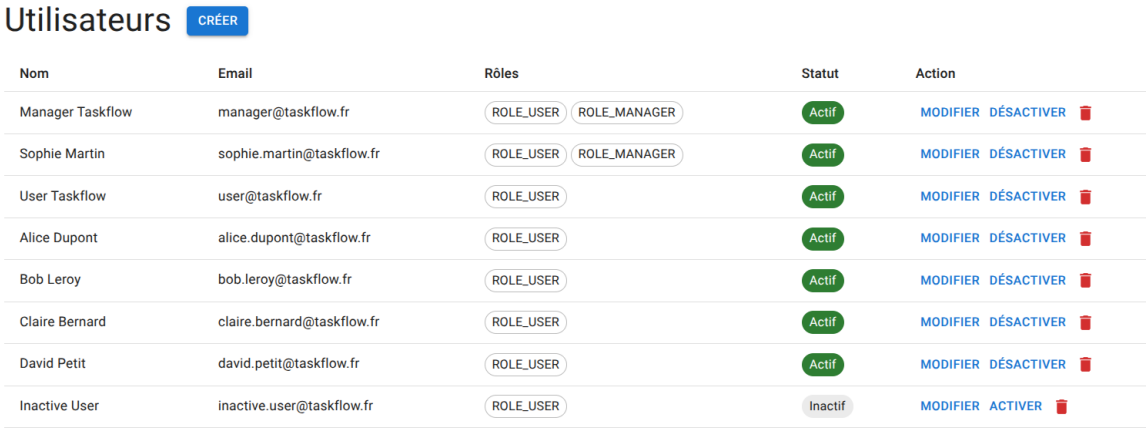




- **Profil :** modification du prénom et du nom. L'email est affiché mais non modifiable, et un message confirme l'enregistrement.



- **Utilisateurs (manager uniquement) :** tableau des utilisateurs (nom, email, rôles), avec création, modification et suppression dans une boîte de dialogue.



Des droits par profils :
Chaque bouton d'action est affiché ou masqué selon les fonctions de **utils/permissions.js**, qui reprennent la matrice d'autorisation. Par exemple :

- un simple membre voit le projet, mais aucun bouton de modification ;
- l'utilisateur assigné à une tâche peut seulement la passer à **ouvert** ou **terminé.** Le menu ne lui propose que ces deux états (**getAllowedTaskStates**) ;
- sur un projet archivé, toutes les actions de modification disparaissent.

Les échanges avec l'API :
Chaque écran récupère ses données au chargement avec **useEffect**, via les fonctions du dossier **api/**. Les filtres de tâches sont envoyés à l'API (**POST /project/{id}/tasks/search**) et la liste se recharge à chaque changement de filtre. J'ai géré les trois états de chaque écran :

- chargement : un composant **Loading** (indicateur de progression) ;

- erreur : le composant **NotifyAlert**, qui affiche le message renvoyé par l'API ;
- succès : l'affichage des données, ou un message de confirmation.

Pendant un envoi, les boutons sont désactivés pour éviter les doubles soumissions.

Composants réutilisables et qualité :
J'ai découpé les grosses pages en composants plus petits (**ProjectTasksSection**, **ProjectTagsSection**, **ProjectMembersSection** ...). Chaque composant reçoit ses données et ses actions par les **props**, vérifiées avec **PropTypes**. Le code est contrôlé par **ESLint**. Les composants principaux sont testés avec **Vitest** et **Testing Library**.

Ergonomie et accessibilité :
Les composants MUI apportent une bonne base :

- les labels sont liés aux champs ;
- les champs obligatoires sont signalés ;
- les alertes et boîtes de dialogue ont les bons rôles **ARIA** ;
- l'attribut **lang="fr"** est défini sur la page.

2. Précisez les moyens utilisés :

- React 19, Vite ;
- Material UI ;
- React Router 7 ;
- PropTypes, ESLint ;
- Vitest, Testing Library ;
- Les maquettes Photopea, comme référence ;
- Documentation officielle de React et de Material UI.

3. Avec qui avez-vous travaillé ?

J'ai développé les interfaces seul, en suivant mes maquettes et la matrice d'autorisation, avec l'aide de la documentation de React et de Material UI.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager – Développement des interfaces de l'application web React**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

- Application en ligne : https://gdu-taskflow-manager-web.onrender.com
- Dépôt GitHub : https://github.com/palepupet/gdu_taskflow_manager_web

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 5 Préparer et exécuter les tests

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Pour l'application web TaskFlow Manager, je voulais vérifier trois choses : que les écrans s'affichent correctement, que les formulaires réagissent bien (saisie, validation, erreurs de l'API), et surtout que les règles d'accès sont respectées dans l'interface. J'ai combiné des tests automatisés, lancés à chaque push, et des tests manuels dans le navigateur.

Mise en place des tests :

J'ai utilisé Vitest, le framework de test intégré à Vite, avec :

- **Testing Library (React)** : affiche un composant et le teste comme un utilisateur le ferait (on cherche un bouton par son texte, un champ par son label…) ;
- **user-event** : simule les vraies actions de l'utilisateur (taper du texte, cliquer) ;
- **jsdom** : simule un navigateur dans le terminal ;
- **jest-dom** : ajoute des vérifications lisibles comme **toBeInTheDocument()** ou **toBeDisabled()**.

La configuration est dans **vite.config.js** (environnement **jsdom** et fichier **src/test/setup.js**). Les tests sont placés dans un dossier '**__tests__**' à côté du code qu'ils testent. Deux scripts npm :

- **npm test** (mode watch pendant le développement) ;
- **npm run test:run** (exécution unique, utilisée par la CI).



```
{
  "name": "gdu_taskflow_manager_web",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "lint": "eslint .",
    "preview": "vite preview",
    "test": "vitest",
    "test:run": "vitest run"
  },
```

J'ai écrit 62 tests, répartis en trois types.

Tests unitaires (utils/) :

Ce sont surtout les permissions, la partie la plus sensible. Pour chaque fonction de **permissions.js**, je teste le cas autorisé et le cas refusé :

- un manager ou l'owner peut modifier un projet actif, pas un utilisateur simple ;
- personne ne peut modifier un projet archivé ;
- l'utilisateur assigné à une tâche ne peut la passer qu'à **ouvert** ou **terminé**, pas à **en cours** ;

Ces cas reprennent directement ma matrice d'autorisation.

Tests de composants (components/) :

- **ProtectedRoute** : affiche le chargement, redirige vers **/login** si l'utilisateur n'est pas connecté, sinon affiche la page ;
- **ManagerRoute** : redirige vers **/projects** si l'utilisateur n'est pas manager ;
- **TaskFormDialog**, **TagFormDialog**, **ProjectForm** : affichage des champs en création et en modification, affichage des erreurs, bouton désactivé pendant l'envoi ;
- **Loading**, **NotifyAlert** : affichage du message et du type d'alerte.

Tests de pages (pages/) :

- **Connexion** : l'utilisateur tape son email et son mot de passe puis clique sur **Se connecter**. Je vérifie que la connexion est appelée avec les bonnes valeurs, puis la redirection vers **/projects**. Si l'API refuse, un message est affiché ;
- **Création de projet** : création réussie avec redirection, erreur si le titre ne contient que des espaces, erreur si la date de fin est avant la date de début, les erreurs sont renvoyées par l'API ;
- **Liste des projets** : chargement, affichage des projets reçus, message en cas d'erreur.

Mocks :

Pour que les tests ne dépendent pas de l'API ni d'un serveur, je remplace certaines dépendances par des mocks avec **vi.mock / vi.fn** :

- le **hook useAuth**, pour simuler un manager, un utilisateur simple ou un visiteur non connecté ;
- les fonctions du dossier **api/**, pour simuler une réponse réussie ou une erreur ;
- **useNavigate** de **React Router**, pour vérifier les redirections.

Exécution des tests :

- **En local** : je lance les tests pendant le développement, et **npm run lint + npm run test:run** avant chaque push.
- Dans la **CI GitHub Actions** : à chaque push, le pipeline enchaîne npm ci, le lint, les tests, le build,

puis le build de l'image Docker. Si un test échoue, le pipeline s'arrête et le problème est visible tout de suite sur GitHub.

```
  npm run test:run

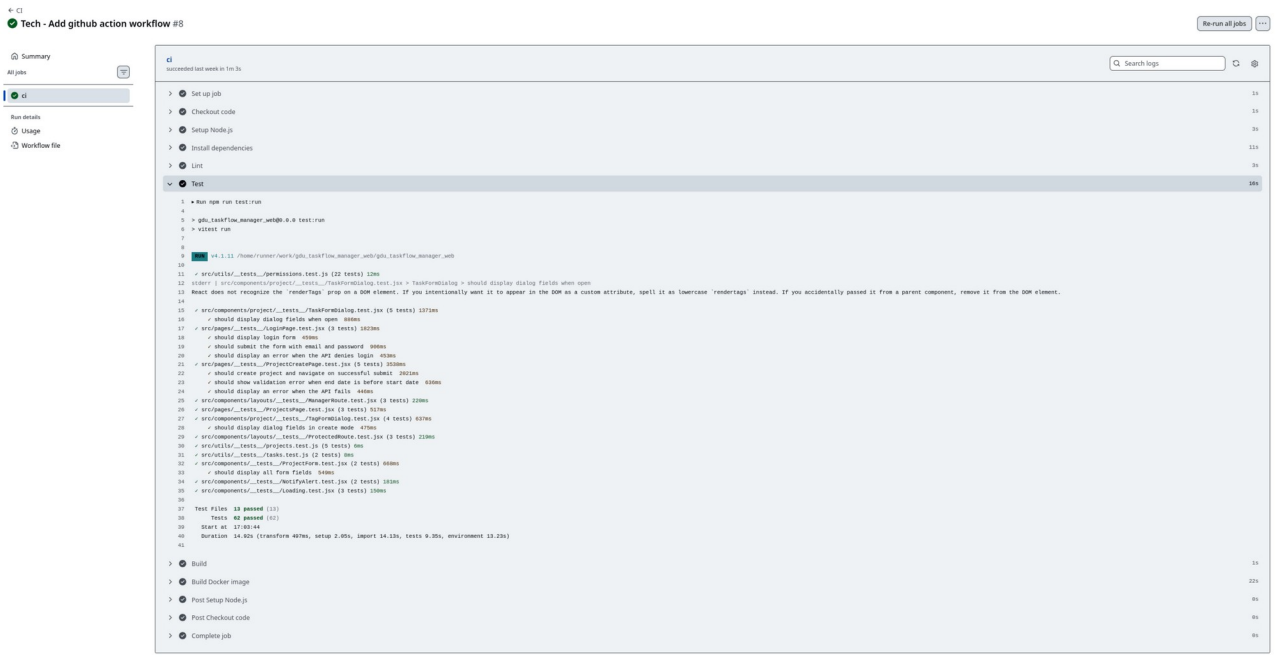
> gdu_taskflow_manager_web@0.0.0 test:run
> vitest run


 RUN  v4.1.11 /home/david/Projects/formation/5_gdu_taskflow_manager_web

 ✓ src/utils/__tests__/projects.test.js (5 tests) 5ms
 ✓ src/utils/__tests__/tasks.test.js (2 tests) 5ms
 ✓ src/utils/__tests__/permissions.test.js (22 tests) 10ms
 ✓ src/components/__tests__/Loading.test.jsx (3 tests) 222ms
 ✓ src/components/layouts/__tests__/ManagerRoute.test.jsx (3 tests) 228ms
 ✓ src/components/__tests__/NotifyAlert.test.jsx (2 tests) 217ms
 ✓ src/components/layouts/__tests__/ProtectedRoute.test.jsx (3 tests) 205ms
 ✓ src/pages/__tests__/ProjectsPage.test.jsx (3 tests) 426ms
 ✓ src/components/__tests__/ProjectForm.test.jsx (2 tests) 459ms
     ✓ should display all form fields  398ms
 ✓ src/components/project/__tests__/TagFormDialog.test.jsx (4 tests) 494ms
     ✓ should display dialog fields in create mode  373ms
stderr | src/components/project/__tests__/TaskFormDialog.test.jsx > TaskFormDialog > should display dialo
React does not recognize the `renderTags` prop on a DOM element. If you intentionally want it to appear i
gs` instead. If you accidentally passed it from a parent component, remove it from the DOM element.

 ✓ src/components/project/__tests__/TaskFormDialog.test.jsx (5 tests) 834ms
     ✓ should display dialog fields when open  542ms
 ✓ src/pages/__tests__/LoginPage.test.jsx (3 tests) 1188ms
     ✓ should display login form  351ms
     ✓ should submit the form with email and password  545ms
 ✓ src/pages/__tests__/ProjectCreatePage.test.jsx (5 tests) 1692ms
     ✓ should create project and navigate on successful submit  1061ms

 Test Files  13 passed (13)
      Tests  62 passed (62)
   Start at  08:25:48
   Duration  5.49s (transform 1.28s, setup 1.89s, import 16.24s, tests 5.98s, environment 20.69s)
```



Tests manuels :

En plus des tests automatisés, j'ai testé l'application dans le navigateur avec des comptes de rôles différents (manager, owner, membre, assigné), créés grâce aux **fixtures** de l'API. J'ai vérifié les parcours principaux et ce que chaque rôle voit. Avec les outils de développement (onglet Réseau), j'ai vérifié les requêtes envoyées, le token et les codes de retour (200, 401, 403…).

2. Précisez les moyens utilisés :

- Vitest 4 : framework de test ;
- Testing Library (React, user-event, jest-dom) ;
- jsdom : navigateur simulé ;
- Mocks Vitest (vi.mock, vi.fn) et MemoryRouter ;
- ESLint ;
- GitHub Actions : exécution automatique à chaque push ;
- Navigateur et outils de développement : tests manuels ;
- Fixtures de l'API : comptes de test par rôle.

3. Avec qui avez-vous travaillé ?

J'ai écrit et exécuté les tests seul, en m'appuyant sur la documentation de Vitest et de Testing Library et sur ma matrice d'autorisation pour choisir les cas à tester.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Tests de l'application web React**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Dépôt GitHub (tests dans les dossiers **__tests__**) :
https://github.com/palepupet/gdu_taskflow_manager_web

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 6 Préparer et documenter le déploiement

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Une fois l'application web développée et testée, j'ai préparé son déploiement. Je voulais trois choses : qu'elle puisse être installée de la même façon sur n'importe quelle machine, que l'URL de l'API soit configurable selon l'environnement, et qu'une autre personne puisse lancer le projet seule en suivant la documentation.

Architecture du déploiement :
J'ai d'abord défini où chaque partie de TaskFlow serait hébergée, et je l'ai représenté dans un diagramme de déploiement :

- les deux dépôts GitHub (API et Web), chacun avec sa propre CI GitHub Actions ;
- l'API déployée sur Render sous forme de Web Service Docker ;
- le front React déployé sur Render sous forme de site statique ;
- la base MySQL hébergée chez Alwaysdata.

Build de production :

La commande **npm run build** (Vite) génère une version optimisée de l'application dans le dossier **dist/** : code minifié et fichiers regroupés. Ce dossier est la seule chose à déployer. Il n'est pas versionné (**.gitignore**).

Configuration :

La seule configuration du front est l'URL de l'API, dans la variable VITE_API_BASE_URL :

Environnement	Valeur	Localisation
Développement (Vite)	http://localhost:8000	.env / .env.local
Docker local	http://localhost:8000	compose.yaml
Production (Render)	https://gdu-taskflow-manager-web.onrender.com/login	Configuration Render

J'ai aussi vérifié le **CORS** côté API : l'origine du front doit être autorisée dans **CORS_ALLOW_ORIGIN**, sinon le navigateur bloque les requêtes.

Conteneurisation avec Docker

Pour que le front tourne partout de la même manière, je l'ai conteneurisé avec un Dockerfile multi-stage :

- Étape de build (**node:22-alpine**) : **npm ci** installe les versions exactes du **package-lock.json**, puis **npm run build** génère **dist/** avec l'URL de l'API reçue en argument (**ARG VITE_API_BASE_URL**).
- Étape de production (**nginx:alpine**) : on ne garde que le dossier **dist/**, servi par **nginx**.

```
FROM node:22-alpine AS build

WORKDIR /app

COPY package.json package-lock.json ./
RUN npm ci

COPY . .

ARG VITE_API_BASE_URL=http://localhost:8000
ENV VITE_API_BASE_URL=$VITE_API_BASE_URL

RUN npm run build

FROM nginx:alpine

COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Le fichier **.dockerignore** exclut **node_modules, dist**, **.git**, **.idea** et **.env.local** de l'image : le build est plus rapide et aucun fichier local n'est copié par erreur.

Le fichier **compose.yaml** permet de tout lancer en une commande, **docker compose up --build**. Le front est alors disponible sur **http://localhost:3000** (port 3000 de la machine relié au port 80 de nginx).

Vérification du Build :

Avant chaque déploiement, je m'assure que le build fonctionne :

- en local, avec npm run build ;
- dans la CI, qui lance à chaque push le lint, les tests, le build, puis le build de l'image Docker.

Documentation :

J'ai rédigé un **README** complet dans le dépôt. Il contient :

- la présentation du projet et la stack technique ;
- les prérequis (Docker, ou Node et npm, et l'API démarrée) ;
- les variables d'environnement et leur rôle ;
- l'installation avec Docker, étape par étape ;
- l'installation sans Docker (npm install, npm run dev) ;
- un récapitulatif des URL (front, API, documentation Swagger) ;
- les comptes de démo par rôle, pour tester rapidement ;
- les routes de l'application et les permissions.

2. Précisez les moyens utilisés :

- Vite : build de production ;
- Docker (Dockerfile multi-stage node:22-alpine puis nginx:alpine), Docker Compose ;
- nginx : serveur des fichiers statiques ;
- GitHub Actions : vérification du build à chaque push ;
- Render : hébergement prévu pour le front et l'API ; Alwaysdata : base MySQL ;
- UMLet : diagramme de déploiement ;
- Markdown : README.

3. Avec qui avez-vous travaillé ?

J'ai préparé et documenté le déploiement seul, avec l'aide de la documentation officielle de Vite, Docker, nginx et Render. Le choix des hébergeurs s'est fait dans le cadre des consignes de la formation.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Préparation et documentation du déploiement de l'application web**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

- README : https://github.com/palepupet/gdu_taskflow_manager_web#readme
- Application en ligne : https://gdu-taskflow-manager-web.onrender.com

Activité-type 1 Développer la partie front-end d'une application web sécurisée

CP 7 Mettre en production (DevOps)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Le but était de mettre l'application web TaskFlow Manager en ligne, de façon automatisée et fiable. Chaque modification passe par les mêmes étapes de vérification avant d'arriver en production, sans manipulation manuelle sur un serveur. C'est le principe de la démarche DevOps : intégrer, tester et livrer en continu.

Cycle de déploiement :

Chaque fonctionnalité suit le même principe :

- une carte Trello ;
- une branche Git dédiée (feat/..., fix/..., tech/...) ;
- une pull request sur GitHub ;
- la CI vérifie automatiquement le code ;
- le merge dans main ;
- le déploiement automatique sur Render.

Intégration continue (CI) :

J'ai créé un workflow GitHub Actions (**.github/workflows/ci.yml**), lancé à chaque push. Il exécute ces étapes dans l'ordre :

- Installation de **Node.js 22**, avec un **cache npm** pour aller plus vite ;
- **npm ci** : installation des versions exactes du **package-lock.json** ;
- **Lint** : contrôle de la qualité du code ;
- **Tests** : les 62 tests **Vitest** ;
- **Build** : vérifie que l'application compile ;
- **Build de l'image Docker** : vérifie que le conteneur se construit.

Si une étape échoue, le pipeline s'arrête et passe au rouge sur la pull request. Je vois le problème avant de merger.

Déploiement continu (CD) :
Le front est hébergé sur Render, relié directement au dépôt GitHub. Je l'ai configuré ainsi :

- **branche déployée** : main ;
- **commande de build** : npm ci et npm run build ;
- **dossier publié** : dist ;
- **variable d'environnement** : VITE_API_BASE_URL = l'URL de l'API sur Render ;
- **règle de réécriture** : '/* => /index.html', pour que React Router gère toutes les URL ;
- **déploiement automatique** : à chaque merge sur main.

Lien avec l'API de production :

L'API est déployée,de son côté, sur **Render** et la base MySQL est hébergée chez **Alwaysdata**. Pour que le front et l'API communiquent en production, j'ai :

- renseigné l'URL de l'API en production dans **VITE_API_BASE_URL** côté front ;
- ajouté l'URL du front dans **CORS_ALLOW_ORIGIN** côté API.

Vérification de la mise en production :

Après chaque déploiement, je vérifie l'application en ligne :

- connexion avec les comptes de démo ;
- parcours principaux (projets, tâches) ;
- appels à l'API dans l'onglet Réseau des outils de développement.

En cas d'erreur, je consulte les logs de build et d'exécution sur Render.

2. Précisez les moyens utilisés :

- **Git et GitHub :** branches, pull requests ;
- **GitHub Actions :** CI ;
- **Render :** hébergement, déploiement automatique ;
- **Alwaysdata :** base MySQL de production (côté API) ;
- **Docker :** vérification de l'image dans la CI ;
- **Trello :** suivi des tâches.

3. Avec qui avez-vous travaillé ?

J'ai mis en place la CI et le déploiement seul, avec l'aide de la documentation de GitHub Actions et de Render, en suivant les consignes de la formation sur la partie DevOps.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Intégration continue et mise en production de l'application web**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Application en ligne : https://gdu-taskflow-manager-web.onrender.com

Documentation de l'API en ligne : https://gdu-taskflow-manager-api.onrender.com/api/doc

Pipelines : onglet Actions du dépôt https://github.com/palepupet/gdu_taskflow_manager_web

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 1 Gérer le projet

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

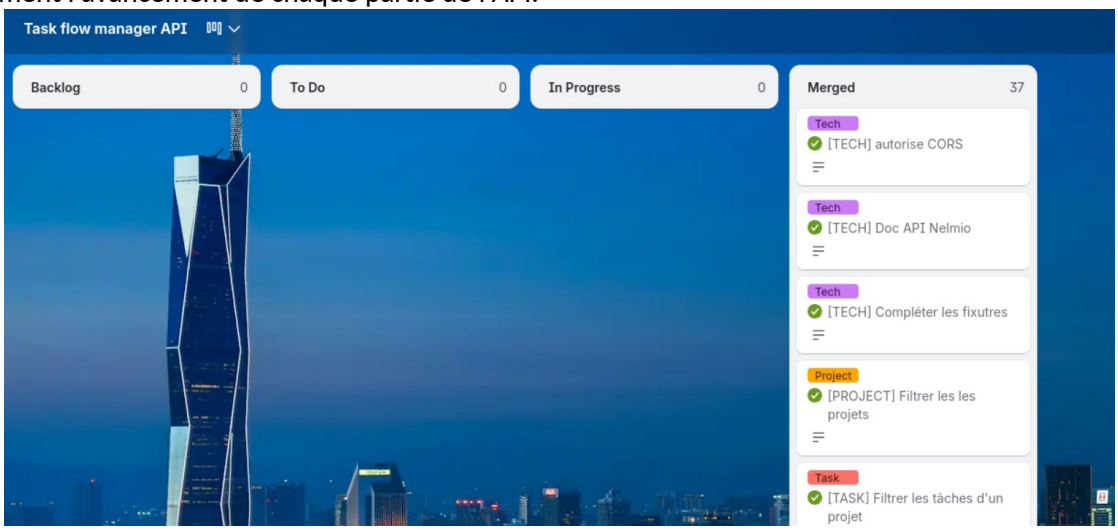
Pour le projet TaskFlow Manager, j'ai organisé mon travail comme dans une équipe de développement, même si j'étais seul. Je voulais savoir à tout moment ce qui restait à faire, avancer petit à petit, par étapes, et garder un historique clair de chaque modification.

Kanban et agilité :

J'ai géré le projet avec un tableau **Kanban** sur **Trello**, organisé en quatre colonnes :

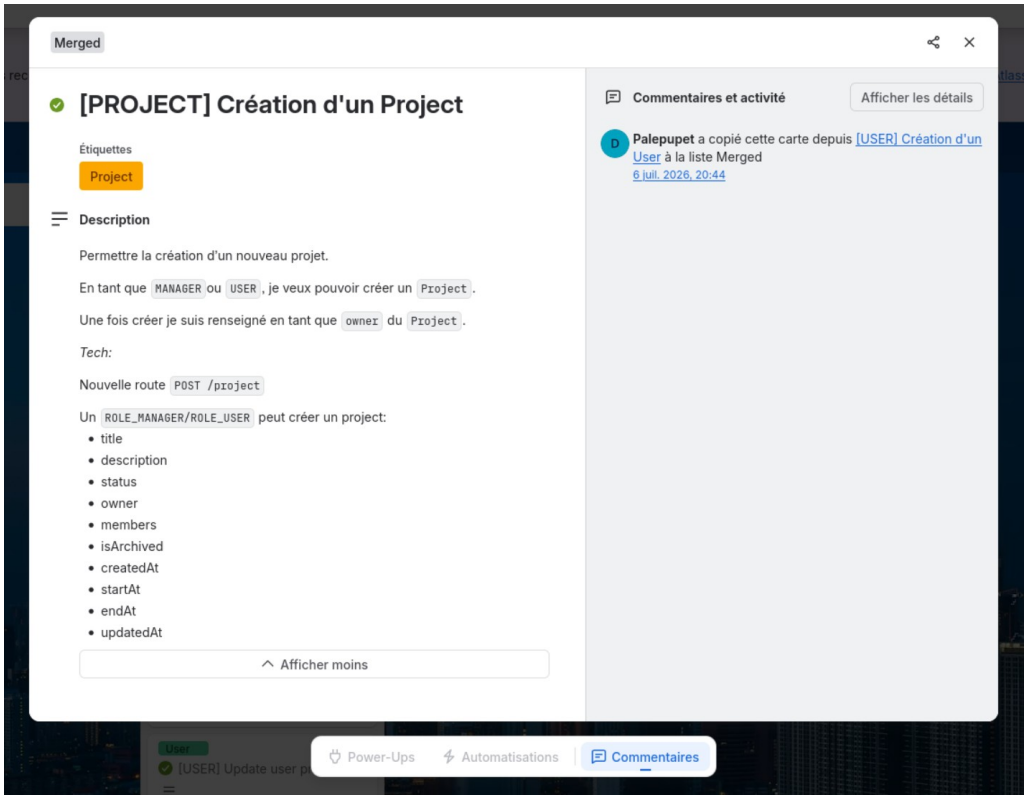
- **Backlog :** toutes les fonctionnalités identifiées ;
- **To Do :** ce que je prévois de faire ensuite ;
- **In Progress :** la carte en cours ;
- **Merged :** terminé.

J'ai classé les cartes avec des étiquettes par domaine : **Connexion**, **User**, **Project**... Je voyais ainsi rapidement l'avancement de chaque partie de l'API.



Users stories :

Chaque carte décrit une fonctionnalité sous forme de **user story**, suivie d'une partie technique. Cette partie technique sert de critères d'acceptation (la route à créer, les rôles autorisés, les filtres attendus...) Elle reprend la matrice d'autorisation. Je savais donc précisément quand une carte était terminée.



Découpage et ordre :
J'ai découpé le projet en différentes cartes et je les ai réalisées dans un ordre logique, chaque étape s'appuyant sur la précédente :

- **Socle technique :** initialisation de **Symfony**, outils de qualité, création de la base ;
- **Utilisateurs et connexion** : entité **User**, **authentification JWT**, **CRUD** des utilisateurs ;
- **Projets :** création, liste, détail, modification, archivage et restauration, lecture seule des projets archivés, membres ;
- **Tâches : CRUD**, assignation, filtres ;
- **Tags : CRUD**, association et dissociation avec les tâches ;
- **Finitions : fixtures** complètes, **CORS**, documentation **Swagger**, **Docker**, **tests**, **CI**, **déploiement.**

Une fois l'API terminée, j'ai développé le front React en suivant la même méthode.

Workflow Trello/Git/Github :
Chaque carte suit le même cycle :

- je déplace la carte dans **In Progress** ;
- je crée une branche dont le nom indique le type de travail : **feat/...** pour une fonctionnalité, **fix/...** pour une correction, **tech/...** pour une tâche technique, **refacto/...** pour un refactoring ;
- je fais des commits avec un message préfixé (**Feature - ...**, **Tech - ...**) ;
- j'ouvre une **pull request** sur **GitHub**. La CI vérifie le code, puis je **merge dans main** ;
- enfin, je déplace la carte dans **Merged**.

Qualité et documentation :
Pour garder un projet stable et maintenable dans la durée :

- je lance **composer check** (**PHPStan**, **PHPMD**, **PHP CS Fixer**, **PHPUnit**) avant chaque push, et la **CI** le relance à son tour ;
- les **README** des deux dépôts expliquent comment installer, configurer et lancer le projet ;
- la documentation **Swagger** de l'API (**/api/doc**) est générée par **NelmioApiDoc**.

2. Précisez les moyens utilisés :

- **Trello :** Kanban, user stories, étiquettes
- **Git :** branches typées, commits préfixés
- **GitHub :** dépôts, pull requests
- **GitHub Actions :** vérification automatique des pull requests
- **Diagrammes de cas d'utilisation et matrice d'autorisation :** base des user stories
- **README et Swagger :** documentation

3. Avec qui avez-vous travaillé ?

J'ai géré le projet seul, à partir du cahier des charges de la formation. Je faisais régulièrement des points d'avancement avec mon formateur, qui me faisait des retours sur les fonctionnalités.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Organisation et suivi du projet (API et Web)**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Tableau Trello : https://trello.com/b/KJUtMTTP/task-flow-manager-api

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 2 Définir l'architecture logicielle

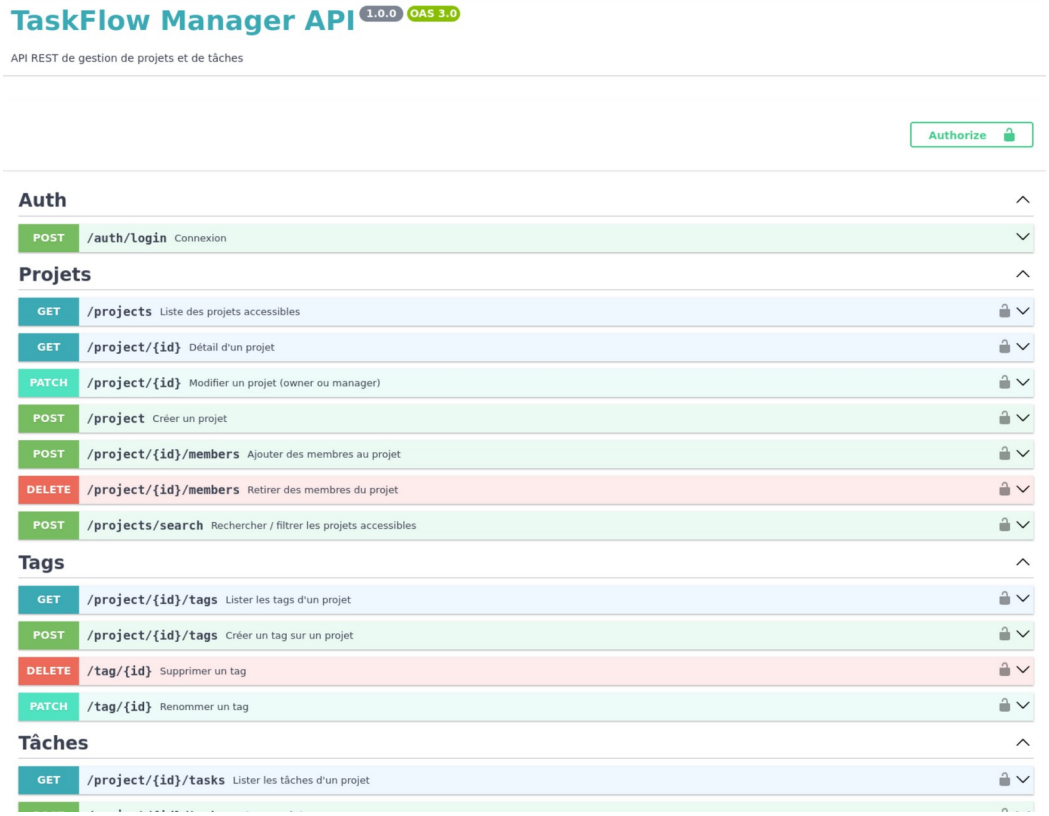
1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'API TaskFlow Manager est le cœur de l'application. Elle porte les données, les règles métier et la sécurité. C'est elle que consomme le front React. Avant de développer les fonctionnalités, j'ai défini comment organiser le code, avec trois objectifs : séparer les responsabilités, garder un code testable, et ne jamais faire confiance aux données reçues.

API REST :

J'ai choisi une architecture REST car :

- chaque ressource a ses URL (**/project**, **/task**, **/tag**, **/users**, **/me...**) ;
- les **verbes HTTP** ont leur sens habituel : **GET** pour lire, **POST** pour créer, **PATCH** pour modifier, **DELETE** pour supprimer ;
- les **codes HTTP** sont précis : 200, 201 (créé), 400 (requête invalide), 401 (non authentifié)... ;
- les données échangées sont uniquement du **JSON**.



Architecture en couches :

J'ai organisé le dossier src/ en couches, chacune avec un seul rôle :

Couche	Dossier	Rôle
Contrôleurs	**Controller/** (Auth, User, Project, Task, Tag)	Reçoivent la requête HTTP, appellent les autres couches et renvoient la réponse JSON.
DTO	**Dto/** (Create-Request, Update-Request, Search-Request)	Décrivent les données en entrée (avec les règles de validation) et en sortie (ce qui est exposé au client).
Entités	**Entity/** (User, Project, Task, Tag)	Représentent les données ET portent les règles métier.
Enum	**Enum/** (ProjectStatus, TaskState…)	Limitent les valeurs possibles (états, priorités, rôles).
Accès aux données	**Repository/** + **interfaces**	Requêtes vers la base, via Doctrine.
Sécurité	**Security/ (**UserChecker, security.yaml)	Authentification JWT, refus des comptes inactifs.
Outils HTTP	**Http/** (RequestPayloadParser, ApiErrorResponse)	Lecture et validation du JSON, format unique des erreurs.
Données de démo	**DataFixtures/**	Jeux de données pour le développement et les tests.

La requête dans son ensemble :

Exemple avec la création d'une tâche, **POST /project/{id}/tasks** :

- Symfony vérifie le token JWT. Sans token valide, l'API répond 401 ;
- Le contrôleur récupère l'utilisateur connecté (**CurrentUserTrait**) et le projet via le repository. Si le projet n'existe pas, l'API répond 404 ;
- L'entité vérifie la règle métier **$project->canCreateTaskBy($user)**. Si l'utilisateur n'a pas le droit, l'API répond 403 ;

- Le JSON est décodé, puis transformé en **DTO** (**CreateTaskRequest**) et validé par le **Validator** (titre obligatoire, date valide, priorité autorisée…). En cas d'erreur, l'API répond 400 avec un message clair ;
- L'entité **Task** est créée. Si un utilisateur assigné est indiqué, **addMemberWhenAssigning()** l'ajoute automatiquement comme membre du projet ;
- **Doctrine** enregistre la tâche (**flush**) ;
- L'API renvoie un **DTO** de réponse (**TaskResponse**) avec le code 201.

La sécurité :
La sécurité est définie dans **security.yaml** avec deux pare-feux :

- login : la route **/auth/login** vérifie l'email et le mot de passe (**json_login**), puis **LexikJWT** génère le token ;
- api : toutes les autres routes exigent un token valide (**IS_AUTHENTICATED_FULLY**).

```yaml
firewalls:
    dev:
        pattern: ^/(_(profiler|wdt)|css|images|js)/
        security: false

    login:
        pattern: ^/auth/login
        stateless: true
        provider: app_user_provider
        user_checker: App\Security\UserChecker
        json_login:
            check_path: /auth/login
            username_path: email
            password_path: password
            success_handler: lexik_jwt_authentication.handler.authentication_success
            failure_handler: lexik_jwt_authentication.handler.authentication_failure

    api:
        pattern: ^/
        stateless: true
        provider: app_user_provider
        user_checker: App\Security\UserChecker
        jwt: ~

access_control:
    - { path: ^/auth/login, roles: PUBLIC_ACCESS }
    - { path: ^/api/doc, roles: PUBLIC_ACCESS }
    - { path: ^/, roles: IS_AUTHENTICATED_FULLY }
```

security.yaml

Documentation :
Chaque route est documentée avec des attributs **OpenAPI** (**#[OA\Get]**, **#[OA\Post]**…) :

- description,
- paramètres et codes de réponse possibles.

2. Précisez les moyens utilisés :

- **PHP 8.2 :** (typage strict, enums, propriétés readonly, constructeurs promus)
- **Symfony 7.2 :** routing par attributs, injection de dépendances, Security, Validator
- **Doctrine ORM**
- **LexikJWTAuthenticationBundle :** authentification JWT
- **NelmioApiDocBundle :** OpenAPI / Swagger
- **NelmioCorsBundle**
- **PHPStan :** vérification des types et de la cohérence entre couches
- **Diagramme de classes et matrice d'autorisation**

3. Avec qui avez-vous travaillé ?

J'ai défini l'architecture seul, en m'appuyant sur la documentation officielle de Symfony (bonnes pratiques, Security, Validator), de mes expérienes et sur les consignes de la formation.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Architecture de l'API REST Symfony**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Dépôt GitHub de l'API : https://github.com/palepupet/gdu_taskflow_manager_api

Documentation Swagger : https://gdu-taskflow-manager-api.onrender.com/api/doc

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 3 Mettre en place la base de données relationnelle

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'API TaskFlow Manager enregistre les utilisateurs, les projets, les tâches et les tags dans une base MySQL 8. Je voulais un modèle qui respecte les règles métier, qui garantisse l'intégrité des données grâce aux contraintes, et qui puisse évoluer proprement tout au long du projet.

Modélisation des données :

À partir de l'analyse du besoin, j'ai identifié les objets principaux et leurs relations, puis je les ai représentés dans un diagramme de classes UML :

- un utilisateur peut posséder plusieurs projets (owner) ;
- un projet a un seul owner et peut avoir plusieurs membres ;
- un projet contient plusieurs tâches et plusieurs tags. Une tâche et un tag appartiennent à un seul projet ;
- une tâche peut être assignée à un utilisateur ;
- une tâche peut avoir plusieurs tags, et un tag peut être utilisé par plusieurs tâches.

Les valeurs figées sont représentées par des énumérations :

- **statut du projet :** en cours, terminé, annulé ;
- **état de la tâche :** ouvert, en cours, terminé ;
- **priorité :** basse, moyenne, élevée.

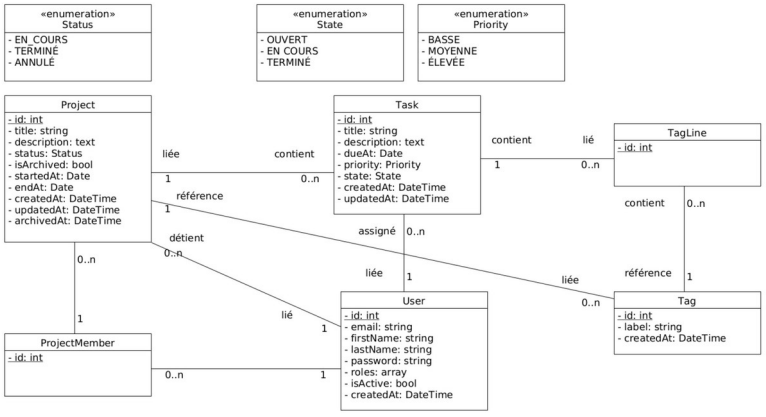


Diagramme de classe

Le modèle physique :

Table	Contenu principal	Clés et contraintes
user	email, mot de passe hashé, prénom, nom, rôles (JSON), actif, date de création.	PK id, email unique.
project	titre, description, statut, archivé, dates de début, de fin et d'archivage.	PK id, FK owner_id => user.
project_member	table de liaison projet <=> membres.	PK (project_id, user_id), FK en ON DELETE CASCADE.
task	titre, description, échéance, priorité, état, dates.	PK id, FK project_id => project, FK assignee_id => user
tag	libellé, date de création.	PK id, FK project_id => project
task_tag	table de liaison tâche <=> tags.	PK (task_id, tag_id), FK en ON DELETE CASCADE

Les deux relations **Many-To-Many** (membres d'un projet, tags d'une tâche) sont gérées par des tables de liaison avec une clé primaire **composite**. Le même membre ne peut donc pas être ajouté deux fois au même projet.

ORM Doctrine :

J'ai traduit ce modèle en entités PHP (**User**, **Project**, **Task**, **Tag**) avec les attributs Doctrine :

- **#[ORM\Column] :** type et taille des colonnes (length: 100 pour un titre, **Types::TEXT** pour une description, unique: true pour l'email…) ;
- **#[ORM\ManyToOne] / #[ORM\OneToMany] :** par exemple, une tâche appartient à un projet ;
- **#[ORM\ManyToMany]** + **#[ORM\JoinTable] :** les tables **project_member** et **task_tag** ;
- **#[ORM\JoinColumn(nullable: false)] :** une tâche ou un tag doit obligatoirement appartenir à un projet ;
- **enumType :** les énumérations PHP sont stockées directement en base ;
- **orphanRemoval: true :** supprimer un projet supprime aussi ses tâches et ses tags ;
- **#[ORM\HasLifecycleCallbacks]** + **#[ORM\PrePersist] / #[ORM\PreUpdate] : createdAt** et **updatedAt** sont remplis automatiquement.

En plus des contraintes en base, j'ai ajouté des contraintes de validation sur les entités, vérifiées avant l'enregistrement :

- **#[Assert\NotBlank]**, **#[Assert\Length]**, **#[Assert\Email]** ;
- au moins un rôle par utilisateur ;
- **#[Assert\Callback]** qui vérifie que la date de fin d'un projet n'est pas avant sa date de début.

```php
#[ORM\Entity(repositoryClass: ProjectRepository::class)]
#[ORM\HasLifecycleCallbacks]
class Project
{
    1 usage
    #[ORM\Id]
    #[ORM\GeneratedValue]
    #[ORM\Column]
    private ?int $id = null;

    2 usages
    #[Assert\NotBlank(message: 'Le titre est obligatoire.')]
    #[Assert\Length(max: 100)]
    #[ORM\Column(length: 100)]
    private ?string $title = null;

    2 usages
    #[ORM\Column(type: Types::TEXT, nullable: true)]
    private ?string $description = null;

    5 usages
    #[ORM\Column(enumType: ProjectStatus::class)]
    private ProjectStatus $status = ProjectStatus::IN_PROGRESS;

    6 usages
    #[ORM\ManyToOne(inversedBy: 'projects')]
    #[ORM\JoinColumn(nullable: false)]
    private ?User $owner = null;
```

*extrait entité **Project.php***

Migrations :

Je n'ai jamais modifié la base de données à la main. Chaque évolution du modèle passe par une migration Doctrine :

- je modifie l'entité ;
- **php bin/console make:migration** génère le SQL correspondant ;
- je relis le fichier généré ;
- **php bin/console doctrine:migrations:migrate** l'applique.

Le projet compte 6 migrations, créées au fur et à mesure des fonctionnalités:

Migration	Contenu
1	table user
2	tables project et project_member
3	colonne archived_at
4	table task
5	table tag
6	table task_tag

Chaque migration possède une méthode **up()** et une méthode **down()**, ce qui permet de revenir en arrière. L'historique de la base est versionné dans Git avec le code, donc n'importe quel environnement (local, Docker, production) peut être remis au même niveau avec une seule commande.

```php
final class Version20260701194703 extends AbstractMigration
{
    no usages  ONGARO David
    public function getDescription(): string
    {
        return 'Create User entity';
    }

    ONGARO David
    public function up(Schema $schema): void
    {
        // this up() migration is auto-generated, please modify it to your needs
        $this->addSql( sql: 'CREATE TABLE user (id INT AUTO_INCREMENT NOT NULL, email VARCHAR(255) NOT NULL, password VARCHAR(255
    }

    ONGARO David
    public function down(Schema $schema): void
    {
        // this down() migration is auto-generated, please modify it to your needs
        $this->addSql( sql: 'DROP TABLE user');
    }
}
```

extrait migration entité User

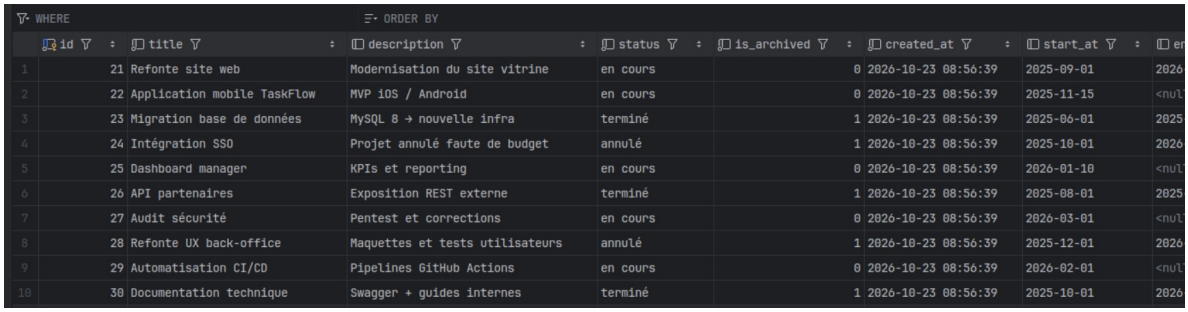
Fixtures :

Pour développer et faire des démonstrations, j'ai créé des fixtures (**UserFixtures**, **ProjectFixtures**, **TaskFixtures**, **TagFixtures**). Elles créent :

- des managers ;

- des utilisateurs, dont un compte inactif ;
- des projets avec des membres ;
- des tâches assignées, avec des tags.

Les mots de passe sont **hashés** avec le **UserPasswordHasher** de Symfony. La commande **php bin/console doctrine:fixtures:load** remet la base dans un état connu.



	id	title	description	status	is_archived	created_at	start_at	en
1	21	Refonte site web	Modernisation du site vitrine	en cours	0	2026-10-23 08:56:39	2025-09-01	2026
2	22	Application mobile TaskFlow	MVP iOS / Android	en cours	0	2026-10-23 08:56:39	2025-11-15	<nul
3	23	Migration base de données	MySQL 8 → nouvelle infra	terminé	1	2026-10-23 08:56:39	2025-06-01	2025
4	24	Intégration SSO	Projet annulé faute de budget	annulé	1	2026-10-23 08:56:39	2025-10-01	2026
5	25	Dashboard manager	KPIs et reporting	en cours	0	2026-10-23 08:56:39	2026-01-10	<nul
6	26	API partenaires	Exposition REST externe	terminé	1	2026-10-23 08:56:39	2025-08-01	2025
7	27	Audit sécurité	Pentest et corrections	en cours	0	2026-10-23 08:56:39	2026-03-01	<nul
8	28	Refonte UX back-office	Maquettes et tests utilisateurs	annulé	1	2026-10-23 08:56:39	2025-12-01	2026
9	29	Automatisation CI/CD	Pipelines GitHub Actions	en cours	0	2026-10-23 08:56:39	2026-02-01	<nul
10	30	Documentation technique	Swagger + guides internes	terminé	1	2026-10-23 08:56:39	2025-10-01	2026

*extrait table **project** depuis l'outil **Database** de PhpStorm*

Sécurité des données :

- les mots de passe sont **hashés** (algorithme auto de Symfony), jamais stockés en clair ;
- les identifiants de la base sont dans **DATABASE_URL**, jamais dans le code versionné (**.env.local**, **.env.docker**, **variables d'environnement Render**) ;
- les requêtes passent par **Doctrine** avec des **paramètres liés**, ce qui protège des i**njections SQL**.

Environnements :

Environnement	Base
Développement	MySQL 8 dans Docker (port 3307 exposé pour PhpStorm)
Tests	SQLite en mémoire, recréée à chaque lancement de tests, rapide et isolée
Production	MySQL hébergé chez Alwaysdata

2. Précisez les moyens utilisés :

- **MySQL 8** (Docker en local, Alwaysdata en production), SQLite pour les tests
- **Doctrine ORM** et **DoctrineMigrationsBundle**

- **DoctrineFixturesBundle**, **MakerBundle**
- **Symfony Validator**
- **UMLet :** diagramme de classes
- **Outil Database de PhpStorm** : visualisation des tables et du schéma

3. Avec qui avez-vous travaillé ?

J'ai conçu et mis en place la base seul, à partir de l'analyse du besoin et de la documentation de Doctrine. Le modèle a été validé avec mon formateur

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Conception et mise en place de la base de données**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Migrations : dossier migrations/ du dépôt https://github.com/palepupet/gdu_taskflow_manager_api

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 4 Développer les composants d'accès aux données SQL

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Une fois la base en place, j'ai développé les composants qui lisent et écrivent les données. Je voulais trois choses : regrouper les requêtes au même endroit, ne renvoyer à chaque utilisateur que les données qu'il a le droit de voir, et protéger l'API contre les injections SQL.

Repositories :

Tout l'accès aux données passe par des r**epositories Doctrine**, un par entité : **UserRepository**, **ProjectRepository**, **TaskRepository** et **TagRepository**. Ils héritent de **ServiceEntityRepository**, qui fournit déjà les méthodes de base (**find, findBy, findOneBy**). J'y ai ajouté mes propres méthodes de recherche.

Pour **Project**, **Task** et **Tag**, j'ai défini une interface (**ProjectRepositoryInterface**) qui liste les méthodes disponibles. Les contrôleurs dépendent de l'interface, pas de la classe concrète. Je peux ainsi changer l'implémentation sans toucher aux contrôleurs.

```php
interface TaskRepositoryInterface
{
    /**
     * @return list<Task>
     */
    1 usage  1 implementation  David
    public function findByProjectId(int $projectId): array;

    5 usages  1 implementation  David
    public function findById(int $id): ?Task;

    /**
     * @return list<Task>
     */
    1 usage  1 implementation  David
    public function searchByProjectId(int $projectId, SearchTaskRequest $criteria): array;
}
```

```php
class TaskRepository extends ServiceEntityRepository implements TaskRepositoryInterface
{
    no usages  David
    public function __construct(ManagerRegistry $registry)
    {
        parent::__construct($registry, Task::class);
    }

    1 usage  David
    public function findByProjectId(int $projectId): array
    {
        /** @var list<Task> $tasks */
        $tasks = $this->createQueryBuilder( alias: 'task')
            ->andWhere( ...where: 'IDENTITY(task.project) = :projectId')
            ->setParameter( key: 'projectId', $projectId)
            ->orderBy( sort: 'task.id', order: 'DESC')
            ->getQuery()
            ->getResult();

        return $tasks;
    }
```

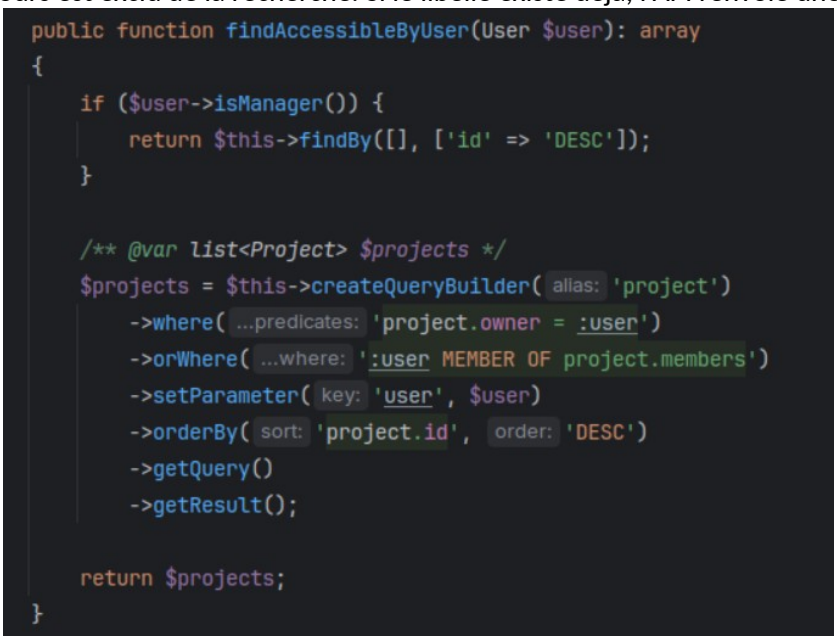
Implémentation de TaskRepositoryInterface.php

QueryBuilder :

Pour les requêtes plus complexes, j'utilise le **QueryBuilder** de Doctrine. Il génère du **DQL**, un langage proche du SQL qui travaille sur les entités. Puis Doctrine le traduit en SQL pour MySQL ou SQLite. Exemples de requêtes que j'ai écrites :

- Projets accessibles à un utilisateur (**findAccessibleByUser**) : un manager voit tous les projets. Les autres ne voient que les projets dont ils sont owner ou membre. Le filtre est appliqué directement dans la requête SQL ;
- Tâches d'un projet (**findByProjectId**) : filtre sur le projet, avec un tri du plus récent au plus ancien ;
- Unicité d'un tag dans un projet (**isTagAlreadyExistsWithThisLabel**) : vérifie qu'un libellé n'existe pas déjà dans le projet, avec un setMaxResults(1) pour ne récupérer qu'une ligne. En modification,

le tag en cours est exclu de la recherche. Si le libellé existe déjà, l'API renvoie une erreur.



```php
public function findAccessibleByUser(User $user): array
{
    if ($user->isManager()) {
        return $this->findBy([], ['id' => 'DESC']);
    }

    /** @var list<Project> $projects */
    $projects = $this->createQueryBuilder( alias: 'project')
        ->where( ...predicates: 'project.owner = :user')
        ->orWhere( ...where: ':user MEMBER OF project.members')
        ->setParameter( key: 'user', $user)
        ->orderBy( sort: 'project.id', order: 'DESC')
        ->getQuery()
        ->getResult();

    return $projects;
}
```

Exemple de QueryBuilder avec du DQL

Recherche à plusieurs critères :

La partie la plus riche est la recherche de tâches (**searchByProjectId**), utilisée par la barre de filtres du front (**POST /project/{id}/tasks/search**). La requête est construite au fur et à mesure, selon les critères reçus :

- **état(s) et priorité(s)** : IN (:states), IN (:priorities) ;
- **échéance avant une date :** :dueAt ≤ :dueBefore ;
- **tags :** une jointure innerJoin('task.tags', 'tag') avec DISTINCT, pour qu'une tâche ayant plusieurs tags ne soit pas renvoyée en double ;
- **assigné :** IDENTITY(task.assignee) = :assigneeId.

Puis vient le tri (champ et ordre). La recherche de projets (**searchByUser**) fonctionne de la même façon : filtres sur le statut et l'archivage, et toujours la restriction **owner/membre** pour les non-managers.

Les critères arrivent dans un DTO (**SearchTaskRequest**) qui a déjà été validé. Le repository ne reçoit donc jamais de données brutes.

```php
public function searchByProjectId(int $projectId, SearchTaskRequest $criteria): array
{
    $query = $this->createQueryBuilder( alias: 'task')
        ->andWhere( ...where: 'IDENTITY(task.project) = :projectId')
        ->setParameter( key: 'projectId', $projectId);

    if (!empty($criteria->state)) {
        $query->andWhere( ...where: 'task.state IN (:states)')
            ->setParameter( key: 'states', $criteria->state);
    }

    if (!empty($criteria->priority)) {
        $query->andWhere( ...where: 'task.priority IN (:priorities)')
            ->setParameter( key: 'priorities', $criteria->priority);
    }

    if (null !== $criteria->dueBefore) {
        $query->andWhere( ...where: 'task.dueAt IS NOT NULL')
            ->andWhere( ...where: 'task.dueAt <= :dueBefore')
            ->setParameter(
                key: 'dueBefore',
                new \DateTimeImmutable( datetime: $criteria->dueBefore.' 23:59:59'),
            );
    }

    if (!empty($criteria->tags)) {
        $query->innerJoin( join: 'task.tags', alias: 'tag')
            ->andWhere( ...where: 'tag.id IN (:tagIds)')
            ->setParameter( key: 'tagIds', $criteria->tags)
            ->distinct();
    }
```

*Extrait **searchByProjectId()** pour la recherche multi-critères*

Injections SQL :

- Toutes les valeurs venant de l'utilisateur passent par des paramètres liés (**:projectId**, **:states**, **setParameter(...)**), jamais par concaténation. Doctrine utilise des **requêtes préparées** (PDO), donc une valeur ne peut pas être interprétée comme du SQL.
- Le tri est le seul cas particulier : un nom de colonne dans ORDER BY ne peut pas être passé en paramètre. J'ai donc mis une liste blanche : seuls **id, dueAt, priority, state, createdAt et title** sont acceptés, et l'ordre doit valoir ASC ou DESC. Toute autre valeur est remplacée par le tri par défaut.

Écriture des données :

Les créations, modifications et suppressions passent par l'**EntityManager** de Doctrine :

- **persist()** prépare l'enregistrement ;
- **flush()** envoie toutes les requêtes en une seule transaction. Si une erreur survient, rien n'est enregistré à moitié ;
- **remove()** supprime.

pour les relations, j'utilise les méthodes des entités (**addMember()**, **addTag()**, **removeTag()...**), qui mettent

à jour les deux côtés de la relation. Doctrine gère ensuite les tables de liaison **project_member** et **task_tag**.

Vérifications :

Les requêtes sont couvertes par les **tests fonctionnels** PHPUnit, qui tournent sur une base **SQLite** en mémoire : filtres, tri, accès selon le rôle.

PHPStan vérifie les types de retour, grâce aux annotations **@return list<Task>**. Pendant le développement, j'ai aussi utilisé le **Profiler Symfony** et **l'outil Database** de PhpStorm pour contrôler le SQL généré.

2. Précisez les moyens utilisés :

- **Doctrine ORM :** repositories, QueryBuilder, DQL, EntityManager
- **PDO** et requêtes préparées
- **MySQL 8**, **SQLite** (tests)
- **PHPUnit**, **PHPStan** avec l'extension Doctrine
- **Bruno :** tests des routes de recherche
- **Outil Database de PhpStorm**

3. Avec qui avez-vous travaillé ?

J'ai développé les composants d'accès aux données seul, en m'appuyant sur la documentation de Doctrine (QueryBuilder, DQL) et sur les consignes de la formation

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Composants d'accès aux données de l'API**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Dépôt : https://github.com/palepupet/gdu_taskflow_manager_api (dossier src/Repository)

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 5 Développer les composants métier

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

Les composants métier traduisent en code les règles de fonctionnement de TaskFlow : qui peut voir quoi, qui peut modifier quoi, comment un projet ou une tâche change d'état. Je voulais que ces règles soient écrites une seule fois, appliquées partout de la même façon, et impossibles à contourner, même en appelant l'API directement sans passer par le front.

Règles du projet :

J'ai placé les règles dans l'entité **Project**, sous forme de méthodes qui répondent à une question précise :

Méthode	Règle
isAccessibleBy($user)	Le manager voit tout, sinon il faut être owner ou membre.
isOwner($user)	L'utilisateur est-il le propriétaire du projet ?
isEditable()	Le projet est modifiable seulement s'il est en cours et non archivé.
canBeModifiedBy($user)	Projet modifiable et utilisateur manager ou owner.
canManageMembersBy($user)	Mêmes conditions que la modification.
canCreateTaskBy / canCreateTagBy / canUpdateTagBy / canDeleteTagBy	S'appuient sur canBeModifiedBy().
canBeRestoredBy($user)	Projet archivé et utilisateur manager ou owner

Les méthodes s'appuient les unes sur les autres. Par exemple, **canCreateTaskBy()** appelle **canBeModifiedBy()**, qui appelle **isEditable()**. Si une règle change (par exemple qui peut modifier un projet), je la modifie à un seul endroit et toutes les actions qui en dépendent suivent.

```php
public function isAccessibleBy(User $user): bool
{
    $project = $this->project;

    return $project instanceof Project && $project->isAccessibleBy($user);
}

4 usages  David
public function canBeModifiedBy(User $user): bool
{
    $project = $this->project;

    return $project instanceof Project && $project->canBeModifiedBy($user);
}
```

Exemple de règles

La vie d'un projet :

Le statut d'un projet est une énumération (**ProjectStatus :** en cours, terminé, annulé). La méthode **changeStatus()** applique la règle métier :

- passer un projet en terminé ou annulé l'archive automatiquement (isArchived = true) et enregistre la date d'archivage ;
- le remettre en cours le restaure : isArchived = false et la date d'archivage est effacée.

Un projet archivé devient en lecture seule pour tout le monde, y compris le manager, puisque **isEditable()** renvoie alors **false**. La méthode **restore()** lève une exception si on essaie de restaurer un projet qui n'est pas archivé. L'objet ne peut jamais se retrouver dans un état incohérent.

```php
public function restore(): void
{
    if (!$this->isArchived()) {
        throw new \LogicException( message: 'Impossible de restaurer un projet non archivé.');
    }

    $this->changeStatus( status: ProjectStatus::IN_PROGRESS);
}

3 usages  David
public function changeStatus(ProjectStatus $status): void
{
    if (ProjectStatus::COMPLETED === $status || ProjectStatus::CANCELLED === $status) {
        $this->status = $status;
        $this->isArchived = true;
        $this->setArchivedAt(new \DateTimeImmutable());

        return;
    }

    $this->status = ProjectStatus::IN_PROGRESS;
    $this->isArchived = false;
    $this->setArchivedAt( archivedAt: null);
}
```

*Méthodes **restore()** et **changeStatus()***

Règles des tâches :

Dans l'entité Task :

- **isAccessibleBy()** et **canBeModifiedBy()** reprennent les règles du projet auquel la tâche appartient ;
- **canChangeStateBy($user, $targetState)** gère un cas particulier, l'utilisateur assigné à une tâche peut changer son état sans pouvoir modifier le reste. Mais seulement vers **ouvert** ou **terminé**. Le passage à **en cours** reste réservé au manager et à l'owner ;
- **canTransitionTo()** refuse une transition vers l'état actuel.

```php
public function canChangeStateBy(User $user, TaskState $targetState): bool
{
    $project = $this->project;
    if (!$project instanceof Project || !$project->isEditable()) {
        return false;
    }

    if ($user->isManager()) {
        return true;
    }

    if ($project->isOwner($user)) {
        return true;
    }

    $assignee = $this->assignee;
    if ($assignee instanceof User && $assignee->getId() === $user->getId()) {
        return TaskState::IN_PROGRESS !== $targetState;
    }

    return false;
}
```

*méthode **canChangeStateBy()***

Autres règles métier :

- **Ajout automatique d'un membre :** quand on assigne une tâche à quelqu'un qui n'est pas membre du projet, **addMemberWhenAssigning()** l'ajoute automatiquement aux membres. L'owner n'est jamais ajouté comme membre.
- **Cohérence des tags :**
  - un tag ne peut être associé qu'à une tâche du même projet ;
  - il ne peut pas être associé deux fois à la même tâche ;
  - deux tags d'un même projet ne peuvent pas avoir le même libellé.

- **Membres :** le propriétaire ne peut pas être ajouté comme membre de son propre projet (**canBeAddedAsMember()**).
- **Utilisateurs :**
  - l'email doit être unique ;
  - tout utilisateur a au moins le rôle **ROLE_USER** (**getRoles()**), et **isManager()** identifie les managers ;
  - le **UserChecker** refuse la connexion d'un compte désactivé.
- **Dates :** un **#[Assert\Callback]** sur Project vérifie que la date de fin n'est pas avant la date de début ;
- **Valeurs par défaut :** une nouvelle tâche est **ouverte** avec une priorité **moyenne**. Un nouveau projet est **en cours** et non archivé. Les dates **createdAt** et **updatedAt** sont remplies automatiquement par les **callbacks Doctrine**.

Les énumérations :

Les **états**, **priorités**, **statuts** et **rôles** sont des **Enums** (**TaskState**, **TaskPriority**, **ProjectStatus**, **UserRole**). Chacune a une méthode **values()**, réutilisée dans les contraintes de validation (**Assert\Choice(callback: ...)**).

Une valeur inconnue est refusée dès l'entrée dans l'API, et le code manipule des constantes typées plutôt que des chaînes de caractères.

Comment les règles sont utilisées :

Dans chaque action, le contrôleur suit le même ordre :

- il récupère l'utilisateur connecté et la ressource ;
- il demande à l'entité si l'action est permise ;
- il valide les données ;
- il vérifie les règles de cohérence ;
- il enregistre.

Les règles sont donc toujours vérifiées côté serveur. Le front n'utilise les mêmes règles que pour afficher ou masquer les boutons.

La vérification :

Les règles métier sont couvertes par les **tests fonctionnels PHPUnit**. Pour chaque route, je teste le cas autorisé et les cas refusés selon le rôle (manager, owner, membre, assigné, utilisateur extérieur, projet archivé).

Ces cas reprennent la matrice d'autorisation.

PHPStan et PHPMD contrôlent la qualité du code.

2. Précisez les moyens utilisés :

- **PHP 8.2 :** enums, typage strict, readonly
- **Symfony 7.2 :** Security (UserChecker, #[IsGranted('ROLE_MANAGER')]), Validator (Assert\Callback, Assert\Choice)
- **Doctrine ORM :** lifecycle callbacks, collections
- **PHPUnit**, **PHPStan**, **PHPMD**
- **Bruno :** vérification manuelle des cas d'erreur
- **Matrice d'autorisation**

3. Avec qui avez-vous travaillé ?

J'ai développé les composants métier seul, à partir du cahier des charges et de ma matrice d'autorisation.

4. Contexte

Nom de l'entreprise, organisme ou association **Global Digital University**

Chantier, atelier, service	**TaskFlow Manager - Règles métier de l'API (droits, cycle de vie des projets et des tâches)**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Dépôt : https://github.com/palepupet/gdu_taskflow_manager_api (src/Entity, src/Enum, src/Security)

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 6 Préparer et exécuter les tests

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'API porte toute la sécurité et les règles métier de TaskFlow. Une erreur à ce niveau peut laisser un utilisateur voir ou modifier des données qui ne le concernent pas. J'ai donc fait des tests, une partie à part entière du développement. Chaque fonctionnalité est livrée avec ses tests, et chaque règle de la matrice d'autorisation est vérifiée par au moins un test.

Stratégie de test :
J'ai choisi des tests fonctionnels. Ils envoient de vraies **requêtes HTTP** à l'API, avec un vrai **token JWT**, et vérifient la réponse. Je teste ainsi toute la chaîne d'un coup :
pare-feu JWT -> contrôleur -> validation -> règles métier -> base de données -> réponse JSON

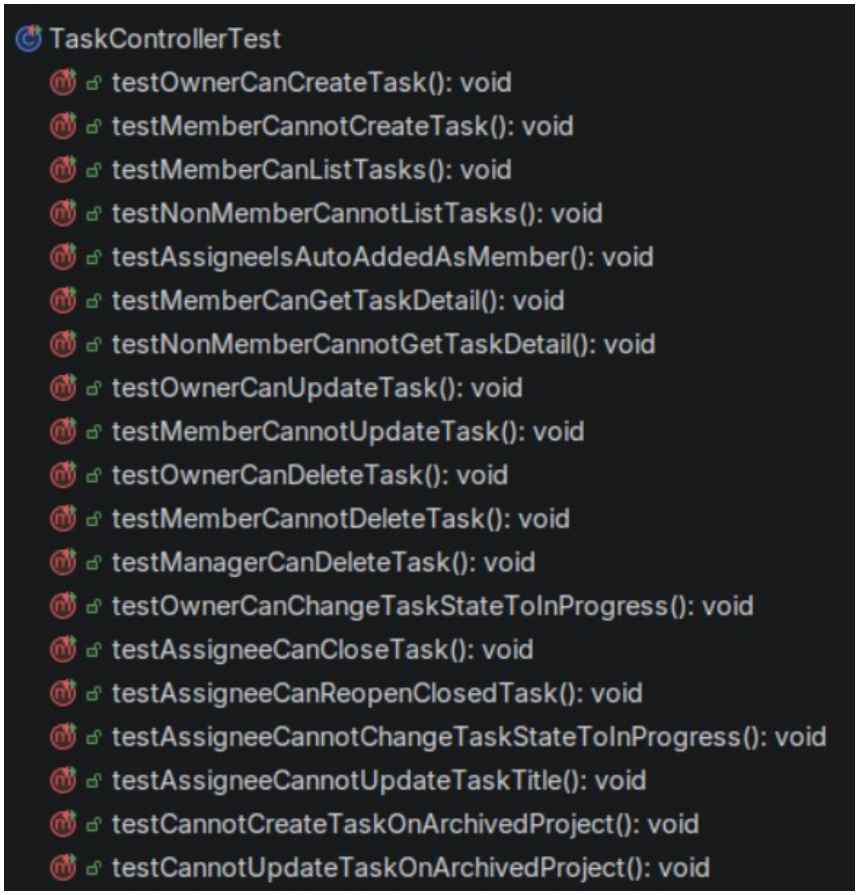
Pour chaque fonctionnalité, je teste systématiquement :

- **le cas nominal :** l'action réussit pour un utilisateur autorisé ;
- l**es refus d'accès :** un utilisateur non autorisé reçoit une 403, et un utilisateur sans token reçoit une 401 ;
- **les données invalides :** titre vide, dates incohérentes… ;
- l**es conflits :** tag en double, tag d'un autre projet… ;
- **les ressources inexistantes :** ressource/page non existante ;
- **le cas du projet archivé :** lecture autorisée, écriture refusée.

J'ai dérivé mes cas de test de la matrice d'autorisation. Une ligne de la matrice (action/rôle) correspond à un test. Les noms des tests décrivent directement la règle vérifiée, ce qui les rend lisibles comme une spécification :

Domaine	Exemples de tests
Authentification	testManagerCanLoginAndReceiveJwtToken, testLoginWithWrongPasswordShouldReturn401, testInactiveUserCannotLogin
Utilisateurs	testManagerCanCreateUserWithManagerRole, testUserCannotListAllUsers, testManagerCanDeactivateUser, testGetMeWithInvalidTokenShouldReturn401
Projets	testUserCanOnlySeesTheirOwnProjects, testMemberCannotUpdateProject, testOwnerCanArchiveProjectBySettingStatusToCompleted, testManagerCannotUpdateArchivedProject, testCannotAddOwnerAsMember, testOwnerCanFilterProjectsByStatus

Tâches	testAssigneeCanCloseTask, testAssigneeCannotChangeTaskStateToInProgress, testAssigneeCannotUpdateTaskTitle, testAssigneeIsAutoAddedAsMember, testCannotAddTagFromOtherProject, testOutsiderCannotSearchTasks
Tags	testCannotCreateDuplicateTag, testMemberCannotCreateTag, testCanListTagsOnArchivedProject



*Extrait liste des méthodes de tests pour **TaskControllerTest.php***

Environnement de test :

- **PHPUnit 11**, avec **WebTestCase** de Symfony qui simule un client HTTP (**KernelBrowser**) ;
- une base **SQLite en mémoire** (.env.test) : rapide, et sans risque pour ma base de développement ;
- une base vide à chaque test : dans **setUp()**, le schéma est supprimé puis recréé (**SchemaTool**). Chaque test est indépendant des autres et part d'un état connu ;
- un **hash de mot de passe** allégé en environnement de test, pour accélérer les tests ;
- une configuration stricte (**phpunit.dist.xml**), **failOnDeprecation**, **failOnWarning** et **failOnNotice** font échouer les tests au moindre avertissement. Ça m'oblige à garder un code propre et à jour.

Chaque test reste ainsi court et lisible, sur le modèle Arrange / Act / Assert (préparer, agir, vérifier).

```php
public function testAssigneeCannotChangeTaskStateToInProgress(): void
{
    ['projectId' => $projectId, 'token' => $ownerToken] = $this->createProjectAs();
    $taskId = $this->addAssignedTaskToProject($projectId, $ownerToken, assigneeEmail: self::EMAIL_ASSIGNEE, ['title' => 'Tâche assignée']);

    $assigneeToken = $this->loginAndGetToken( email: self::EMAIL_ASSIGNEE, password: self::PASSWORD_USER);
    $this->updateTaskById($taskId, ['state' => TaskState::IN_PROGRESS->value], $assigneeToken);

    self::assertResponseStatusCodeSame( expectedCode: Response::HTTP_FORBIDDEN);
}
```

Exemple complet d'un test

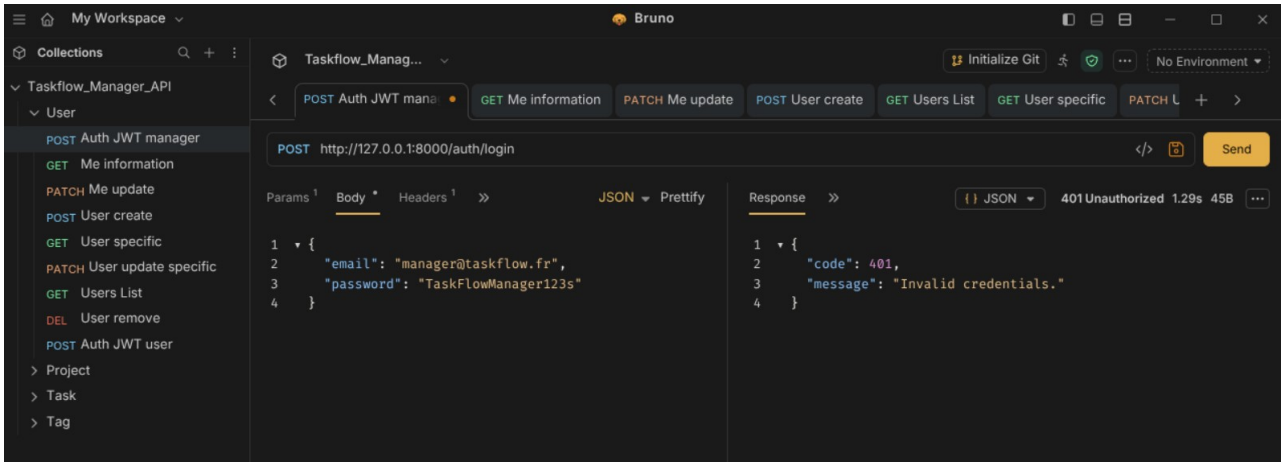
Exécution des tests :

- **En local : composer test** pendant le développement, et surtout **composer check** avant chaque push. Cette commande enchaîne **PHPStan**, **PHPMD**, **PHP CS Fixer** puis **PHPUnit**.
- **Dans la CI GitHub Actions :** à chaque push, le pipeline installe **PHP 8.2 et les dépendances**, génère une paire de **clés JWT de test**, lance **composer check**, puis construit l'image Docker. Une seule erreur bloque le pipeline.

Tests manuels :

En complément, j'ai testé chaque route avec **Bruno** pendant le développement. J'ai créé une collection par ressource, avec la connexion et la récupération du token, pour vérifier les réponses JSON et les codes HTTP.

J'ai aussi utilisé la documentation **Swagger** (**/api/doc**), qui permet d'appeler les routes directement.



requête refusée et remonté de l'erreur

Les tests m'ont permis de refactorer sans crainte. Par exemple, la carte Trello **[TECH] Refacto des tests** a donné la classe **ApiTestCase**. Ils m'ont aussi permis de vérifier que l'ajout d'une fonctionnalité (les tags, par exemple) ne cassait pas les règles existantes (tests de non-régression).

2. Précisez les moyens utilisés :

- **PHPUnit 11**, **WebTestCase** et **KernelBrowser** de Symfony
- **SQLite** en mémoire, **SchemaTool** de Doctrine
- **LexikJWT :** vrais tokens dans les tests
- **Composer scripts** (composer test, composer check)
- **PHPStan**, **PHPMD**, **PHP CS Fixer**
- **GitHub Actions**
- **Bruno**, **Swagger** (**NelmioApiDoc**)
- **Matrice d'autorisation :** base du plan de tests

3. Avec qui avez-vous travaillé ?

J'ai écrit et exécuté les tests seul, en m'appuyant sur la documentation de Symfony (tests fonctionnels) et de PHPUnit, et sur ma matrice d'autorisation pour définir les cas à tester.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Plan de tests et tests fonctionnels de l'API**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

Tests : dossier tests/ du dépôt https://github.com/palepupet/gdu_taskflow_manager_api

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 7 Préparer et documenter le déploiement

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'API doit pouvoir tourner dans plusieurs environnements, sur mon poste, dans Docker, dans la CI et en production. Je voulais qu'elle s'installe toujours de la même façon, que les secrets ne soient jamais dans le code, et qu'une autre personne puisse la lancer seule grâce à la documentation.

Architecture de déploiement :

J'ai défini l'hébergement cible et je l'ai représenté dans un diagramme :

- le code est sur GitHub, avec une **CI GitHub Actions** ;
- l'API est déployée sur **Render** ;
- la base MySQL de production est hébergée chez **Alwaysdata** ;
- le front est sur **Render**.

Conteneurisation de l'API :

J'ai écrit un **Dockerfile** basé sur l'**image officielle php:8.2-apache** :

- installation des extensions PHP nécessaires (**pdo_mysql, zip**) ;
- le **DocumentRoot** d'Apache pointe sur **public/**. Seul le point d'entrée **index.php** est exposé, jamais le code source, la configuration ou le .env ;
- copie du code, installation de **Composer** puis des dépendances avec **--optimize-autoloader** ;
- droits sur le dossier **var/** donnés à l'utilisateur Apache ;
- génération des **clés JWT** (**lexik:jwt:generate-keypair**), pour que le conteneur soit prêt à signer des tokens.

Le fichier **.dockerignore** exclut **vendor**, **var/cache**, **var/log**, **.git**, **.idea**, et surtout **.env.local** et **.env.docker.** Aucun secret local ne se retrouve dans l'image.

```
FROM php:8.2-apache

RUN apt-get update && apt-get install -y libzip-dev \
    && docker-php-ext-install pdo_mysql zip \
    && a2enmod rewrite

ENV APACHE_DOCUMENT_ROOT=/var/www/html/public
RUN sed -ri -e 's!/var/www/html!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/sites-available/*.conf \
    && sed -ri -e 's!/var/www/!${APACHE_DOCUMENT_ROOT}!g' /etc/apache2/apache2.conf /etc/apache2/conf-available/*.conf

WORKDIR /var/www/html
COPY . .

RUN curl -sS https://getcomposer.org/installer | php -- --install-dir=/usr/local/bin --filename=composer \
    && composer install --optimize-autoloader \
    && chown -R www-data:www-data var

RUN mkdir -p config/jwt \
 && php bin/console lexik:jwt:generate-keypair --overwrite --no-interaction
```

Dockerfile de l'api

Docker Compose :

Le fichier **compose.yaml** décrit deux services :

- **database :** MySQL 8, avec un volume **db_data**, pour garder les données même si le conteneur est supprimé ;
- **api :** construite à partir du **Dockerfile** et exposée sur le port 8000. Elle dépend de la base (**depends_on**) et reçoit sa configuration par variables d'environnement.

Tout se lance avec une seule commande **docker compose --env-file .env.docker up --build -d**.

```
docker compose --env-file .env.docker up
[+] up 2/2
 ✔ Container 4_gdu_taskflow_manager_api-database-1 Running
 ✔ Container 4_gdu_taskflow_manager_api-api-1      Running
Attaching to api-1, database-1
```

Conteneurs démarrés

Configuration et secrets :

La configuration est entièrement externalisée dans des variables d'environnement :

Variable	Rôle
APP_ENV / APP_SECRET	Environnement Symfony (dev, test, prod) et clé secrète
DATABASE_URL	Connexion à la base (database dans Docker,

	Alwaysdata en production)
MYSQL_*	Identifiants du conteneur MySQL
JWT_SECRET_KEY / JWT_PUBLIC_KEY / JWT_PASSPHRASE	Clés et phrase de passe du JWT
CORS_ALLOW_ORIGIN	Origines autorisées à appeler l'API (le front)

Les fichiers sont organisés ainsi :

- **.env** est versionné, avec les valeurs vides. Il sert de modèle ;
- **.env.local** et **.env.docker** contiennent les vraies valeurs et sont ignorés par Git ;
- **.env.test** est versionné, pour la configuration des tests ;
- en production, les valeurs sont saisies dans l'interface de Render ;
- les **clés JWT** (**config/jwt/*.pem**) ne sont jamais versionnées.

Mise en place :

J'ai défini les étapes pour qu'une installation fonctionne du premier coup :

- créer le fichier d'environnement (cp .env .env.docker) et le remplir ;
- builder et démarrer les conteneurs ;
- appliquer les migrations (**doctrine:migrations:migrate**) pour créer les tables ;
- générer les **clés JWT** si besoin ;
- charger les **fixtures** (facultatif) ;
- vérifier : documentation **Swagger** sur **/api/doc**, logs du conteneur, connexion avec un compte de démo.

Avant déploiement :

Avant toute mise en ligne, la CI vérifie la qualité (**PHPStan**, **PHPMD**, **PHP CS Fixer**), lance les tests et construit l'image Docker. Une image qui ne se construit pas est détectée avant d'arriver sur Render.

Documentation :

J'ai rédigé un README complet, il contient :

- la stack technique et les prérequis (avec ou sans Docker) ;
- le rôle de chaque fichier d'environnement (versionné ou non) ;
- l'installation avec Docker, étape par étape, et l'installation sans Docker ;
- les commandes utiles (console Symfony dans le conteneur, client MySQL, arrêt avec ou sans suppression des données) ;
- l'accès à la base depuis l'hôte ;
- les comptes de démo (fixtures) ;
- l'authentification (obtenir et utiliser le token), les rôles, toutes les routes avec des exemples de corps JSON, et les valeurs des enums.

En complément, la documentation **Swagger** (**NelmioApiDoc**) est générée automatiquement à partir du code et accessible sur **/api/doc**.

2. Précisez les moyens utilisés :

- Docker, Docker Compose
- Apache
- Composer
- Doctrine Migrations, DoctrineFixtures
- LexikJWT : génération des clés
- Variables d'environnement, fichiers .env*, .gitignore, .dockerignore
- GitHub Actions : build de l'image dans la CI
- Render, Alwaysdata : hébergement cible
- Markdown (README), NelmioApiDoc (Swagger), UMLet (diagramme)

3. Avec qui avez-vous travaillé ?

J'ai préparé et documenté le déploiement seul, avec l'aide de la documentation de Symfony (déploiement, variables d'environnement), de Docker et de Render, dans le cadre des consignes de la formation.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Conteneurisation, configuration et documentation du déploiement de l'API**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

README : https://github.com/palepupet/gdu_taskflow_manager_api#readme

Swagger : https://gdu-taskflow-manager-api.onrender.com/api/doc

Activité-type 2 Développer la partie back-end d'une application web sécurisée

CP 8 Mettre en production (DevOps)

1. Décrivez les tâches ou opérations que vous avez effectuées, et dans quelles conditions :

L'objectif était de mettre l'API TaskFlow en ligne avec une démarche DevOps. Chaque modification est vérifiée automatiquement puis déployée sans manipulation manuelle sur un serveur, avec une configuration de production sécurisée.

La livraison :

Chaque évolution de l'API suit le même chemin :

- une carte Trello ;
- une branche Git (**feat/…**, **fix/…**, **tech/…**) ;
- une pull request ;
- la **CI GitHub Actions** ;
- le merge dans main ;
- le déploiement automatique sur Render.

Intégration continue (CI) :

Le workflow .github/workflows/ci.yml se déclenche à chaque push et enchaîne :

- Checkout du code ;
- Installation de PHP 8.2 avec les extensions nécessaires (dont **pdo_sqlite** pour les tests) et **Composer 2**, avec un cache des dépendances ;
- composer install ;
- Génération d'une paire de clés JWT de test : les vraies clés ne sont jamais dans le dépôt, donc la CI crée les siennes ;
- composer check :
  - **PHPStan** (analyse statique) ;
  - **PHPMD** (complexité, code mort) ;
  - **PHP CS Fixer** en mode vérification (normes @Symfony et PSR-12) ;
  - **PHPUnit** ;
- Build de l'image Docker, pour vérifier que l'image de production se construit.

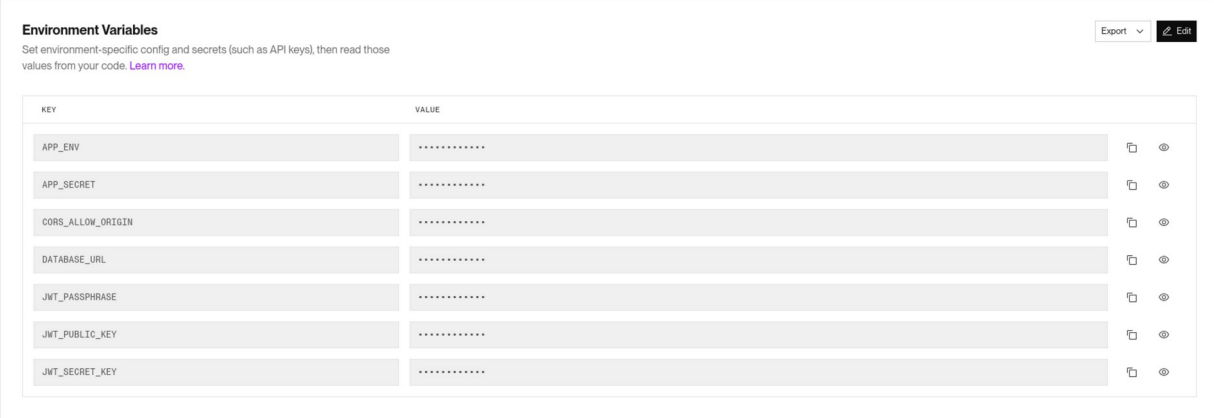
Si une seule étape échoue, le pipeline passe au rouge et je ne merge pas.

Déploiement continu (CD) :

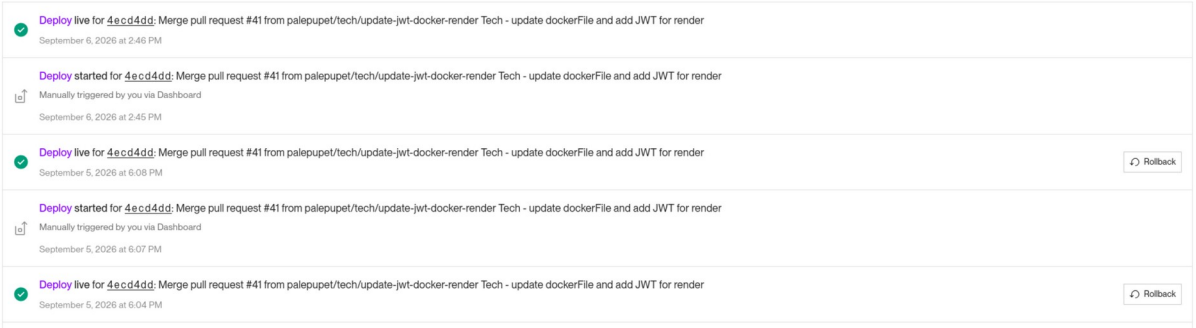
L'API est déployée sur Render en Web Service Docker, relié au dépôt GitHub :

- **branche déployée :** main ;
- **build :** Render construit l'image à partir de mon Dockerfile (le même que celui vérifié par la CI) ;
- **déploiement automatique :** à chaque merge ;
- **variables d'environnement :** saisies dans l'interface Render, jamais dans le code (APP_ENV=prod, APP_SECRET, DATABASE_URL, JWT_PASSPHRASE, CORS_ALLOW_ORIGIN).

Le passage à Render a demandé une adaptation, générer les **clés JWT directement dans l'image**, puisqu'elles ne sont pas versionnées.



variable d'environnement sur Render



extrait d'historique de déploiement sur Render

Base de données de production :

La base MySQL de production est hébergée chez **Alwaysdata**. L'API s'y connecte grâce à **DATABASE_URL**, définie dans **Render**. Le schéma a été créé avec les **migrations Doctrine**. J'ai ensuite chargé les données de démonstration (fixtures).

Configuration sécurisée de la production :

- **HTTPS obligatoire :** Render fournit le certificat. Le token JWT et les mots sont donc chiffrés ;
- **APP_ENV=prod :** les erreurs affichent un message générique, sans trace ni détail technique, et le cache Symfony est optimisé ;
- **CORS restreint :** seule l'URL du front en production est autorisée à appeler l'API depuis un navigateur ;
- **secrets :** uniquement dans les variables d'environnement de Render ;
- **routes protégées :** sans token valide, toute route renvoie une 401, sauf **/auth/login** et **/api/doc**.

Après déploiement :

Après chaque déploiement, je vérifie :

- que la documentation **Swagger** répond sur **/api/doc** ;
- la connexion avec un compte de démo, et la récupération du token ;
- une route protégée, sans token, puis avec token ;
- le fonctionnement complet depuis le front en production ;
- les logs du service sur Render en cas de problème.

Rollback :

Si un déploiement pose problème, je peux :

- revenir à un déploiement précédent depuis Render ;
- ou annuler le changement avec un **git revert** sur main, ce qui relance automatiquement la CI puis le déploiement.

2. Précisez les moyens utilisés :

- **Git, GitHub :** branches, pull requests
- **GitHub Actions :** cache Composer, composer check, build Docker
- **Docker :** image php:8.2-apache
- **Render :** variables d'environnement, HTTPS, logs, rollback
- **Alwaysdata :** MySQL de production
- **Doctrine Migrations**

- **Bruno, Swagger :** vérifications après déploiement
- **Trello :** suivi

3. Avec qui avez-vous travaillé ?

J'ai mis en place la CI et le déploiement seul, en suivant les consignes de la formation pour la partie DevOps, avec l'aide de la documentation de GitHub Actions, de Render et de Symfony.

4. Contexte

Nom de l'entreprise, organisme ou association	**Global Digital University**
Chantier, atelier, service	**TaskFlow Manager - Intégration continue et mise en production de l'API**
Période d'exercice	**Du 29/06/2026 au 05/09/2026**

5. Informations complémentaires *(facultatif)*

API en production (documentation) : https://gdu-taskflow-manager-api.onrender.com/api/doc

Pipelines : onglet Actions de https://github.com/palepupet/gdu_taskflow_manager_api

Titres, diplômes, CQP, attestations de formation

(facultatif)

Intitulé	Autorité ou organisme	Date
Chef de projet digital	Institut Européen de Formation en Ingénierie Informatique (IEF2I)	21/06/2017
Communication et industries graphiques	Lycée polyvalent Claude-Garamont	22/06/2011

Déclaration sur l'honneur

Je soussigné(e) .....***ONGARO David***.....,

déclare sur l'honneur que les renseignements fournis dans ce dossier sont exacts et que je suis l'auteur(e) des réalisations jointes.

Fait à ..*Fontenay-Sous-Bois*..... le ...27/10/2026.....

pour faire valoir ce que de droit.

Signature :



Documents illustrant la pratique professionnelle

(facultatif)

Intitulé
Cliquez ici pour taper du texte.

ANNEXES

(Si le RC le prévoit)